



Fra Rådata til Forretningsværdi

Udvikling af et "Proof of concept"-værktøj til GoFact.



Deltagerens Navn	Mark Nielsen
Vejlederens Navn	Flemming Koch Jensen
Måned og år	januar 2026
Omfang	84083 tegn inkl. Mellemrum. 38 normalsider

JANUARY 5, 2025

EAMV - Datamatiker Hovedopgave

Contents

Introduktion	3
Problemformulering	4
Afgræsning	4
Virksomhedsbeskrivelse	5
Problemdomæne	7
Hvordan har vi tænkt os at vælge hvad produktet skal kunne?	9
MoSCoW Modellen	9
Metode Valg	16
Fil struktur	16
Praktisk.....	17
Databærende objekter	18
Komprest data	19
Sorterings Metode.....	19
Teori	21
Regex	21
Min brug af regex opsummeret.....	21
Optimering af regex.....	22
En regex i koden forklaret	23
Valley af Tal / Custom Hasing	27
Procesbeskrivelse	29
Scrum	29
Sprints	29
Retrospektiv.....	30
Kanban Board	30
Vandfald.....	30
Produktbeskrivelse	31
Fil Struktur.....	31
GenerateInputFiles	32
Step 1. Loading Files	34

Step 2. Data Handling.....	36
Step 3. Skrive output	39
Sortering efter størrelse	40
Perspektivering.....	45
Konklusion	45
Bibliografi	46
Bilag.....	46
MoSCoW – Simplet	46

Introduktion

Tøj industrien har meget spild og det er en af de større områder af global opvarmning. En løsning til dette problem kunne være at sige til de individuelle der køber tøj, at de skulle genbruge, og købe mindre tøj. Men det er ikke en god løsning, da det kræver ændringer fra firmaer eller staten for at have en stor nok impact. Problemet kræver en løsning så tæt på starten af kæden som muligt. Fabrikkerne laver tøj efter hvad en butik bestiller, og en butik bestiller efter det data de har. Hvis de har solgt godt på en trøje, vil de bestille flere hjem. Men hvordan ved de hvad der sælger godt? Hvordan ved de hvor mange de skal bestille hjem, for at optimere hvor meget de kan sælge, og hvor lidt de smider ud? Det er et komplekst problem, der kræver en masse dataindsamling, håndtering og kommunikation. Man kunne løse problemet i en butik med en blyant, papir og en masse mandtimer til at tælle hvert produkt, men det bryder ned i en større butik, og som en del af en kæde. Det er der GoFact kommer ind. De håndterer det data tøjbutikker har om deres salg, lager og katalog. Og med de data kommer de med forslag om hvor mange der skal købes af et stykke tøj, og om der eventuelt kunne byttes om på lager mellem butikker i samme kæde, for at mindske spild. Det er nemt for butikker at bruge GoFact, da de har et motto der hedder "come as you are", som betyder at den måde den individuelle butik opbevarer data er godt nok, og man bare skal sende ind hvad man har. Dette betyder at GoFact skriver individualiseret kode til hver butik så deres data virker med deres system. Dette er en kæmpe fordel for butikker, da det er nemt at tilslutte sig GoFacts system, og starte med at være kunder. Men det betyder dog at hvis en butik ikke er sikker på om de kan bruge GoFact, ville GoFact bruge en del tid på at sætte dem op i deres system, og det ville måske gå spildt hvis butikken ikke ville bruge det alligevel. Det er dette problemområde jeg ville fokusere på. Gøre det nemt for kunder at teste GoFacts hovedfunktion, uden at GoFact skal tabe en masse penge på at sætte en kunde op som ikke bliver en fast kunde alligevel.

Problemformulering

Butikker har det problem at de oftest er styret af individuelle personer. Selv i en kædebutik, kan ejeren være en gammel person og vil gøre tingene på deres egen måde. Det er derfor det kan være svært at overbevise en kunde om at de skal bruge GoFact. Oftest er det kæden som GoFact går til, da mindre butikker ikke er værd opsætning hvis de går ud. Men det betyder at der er et marked som de ikke går efter lige nu. Det er dem man kan få ind med en "demo" af produktet. Så min problemformulering lyder:

"Hvordan kan jeg lave et program der hjælper kunder med at se GoFacts' værdi, og hvordan kan jeg lave et program der hjælper GoFact fange et nyt marked uden at brænde for mange penge på det?"

Afgræsning

Der er mange ting man kunne have med for at vise hvad GoFacts' system kan, men det er ikke alle der er vigtige for at sælge produktet til en kunde. GoFact har features som lager tracking, bestilling, "hot-meter" (som er hvor varmt et produkt er. Det er udregnet ud fra hvor lang tid det har været på salg, og hvor mange der er solgt). Disse features er nice-to-haves, og er ikke grunddelen. Så dem har jeg ikke tænkt mig at fokusere på.

Hoved-delen af deres system er optælling af salg. Et resume af hvad der rent faktisk er solgt. Planen for programmet er at få en salgsfil ind, lave noget data håndtering på det og komme med et sorteret resultat, som er en forkortet version af det rod som er en salgsfil. En salgsfil er hvad deres POS system, (point of sale) skriver til ved hver transaktion. Det er "denne produkt, med disse info, blev solgt x antal gange". Hvis det samme produkt bliver solgt to gange på en dag, kommer der to linjer med samme data. Det er dette der kan gøre det svært at læse, da man ikke nemt kan se hvor mange af et slags produkt der blev solgt.

GoFact starter med kundens data. Det er her problemet med deres data som er anderledes kommer ind i billedet. Det er et problem at ændre deres data hvis det skal gøres automatisk, men i denne applikation gør vi det en gang, og det er dermed et mindre problem. I denne applikation tager vi en salgsfil og kører igennem vores system. Det betyder at vi ændre data til at passe hvad programmet forventer. Dette gør at man ikke skal bruge ressourcer til at ændre eller skrive kode som oversætter det for en, men vi "bare" skal gøre det selv. Det er hurtigere at ændre overskrifter i en CSV fil end det er at skrive en oversætter til den. Dette gør også at man ikke skal være bange for at kunden bruger test-systemet i stedet for hoved-systemet, da det ikke er bygget til at være effektivt uden en fast bruger.

Vi holder os også fra datoer. I kundens fil de kommer med kan der være salg for 3 år siden, eller for 2 uger siden. Det ville være svært at give et fuldstændt billede hvis man siger til kunden at det er salg for hele måneden, hvis der kun er salg for 2 uger. Så det sænker forvirring med programmet her hvis vi siger ”Det data der kommer ind, bliver sorteret og talt”.

I slutningen af programmet, skal vi give kunden data igen de kan forstå. Det betyder et hurtigt resume af de forskellige tøj-kategorier der er blevet solgt, og hvor mange penge der er solgt for i hver kategori. Vi giver også et resume per-produkt, men det kan komme til at fylde meget, så vi kommer med et resume per kategori først.

Virksomhedsbeskrivelse

GoFact er som sagt et data håndterings firma, men ikke til AI eller lign. De indsamler data for kunder, sortere og forkorter det, og giver kunder et bedre overblik. De startede med at skulle have noget kode ud af døren hurtigt, og det resulterede i at deres backend er lidt af et rod, og indeholder en del spildte udregninger. Hver gang en ny kunde kommer ind skal deres data også gennemtjekkes for at se om der er nogle mærkelige fejl som de ikke selv har lagt mærke til. Det kan være ting som at have sin delimiter i en CSV fil det samme som ens decimale værdier. Dette ville resultere i et automatisk system som har alt for meget data, og hvor der højst sandsynligt ville være type-mismatches.

Det er komplekse problemer, men det har resulteret i et robust system der tager noget tid at komme ind i, og som oftest er langsommere end det burde være. For de kunder der allerede er inde i systemet, betyder det ikke alt for meget, da de får et resume af for-dagens salg på starten af morgenen.

Det meste af GoFacts’ udviklings-timer kommer fra at udvikle nye funktioner. Deres salgs-model er en abonnement system, som betyder at de fortsat tjener penge på kode de har skrevet før. Dermed giver det mere mening at skrive ny kode, end at opdatere gammel kode hvis det ikke kræver det. Det betyder også at deres system forbliver nyt, da der kommer nye features og andre nice-to-haves som gør at kunder forbliver kunder hos GoFact. Hver kunde har også direkte adgang til enden CTO Morten V.G. eller CEO Henning H. som betyder at når der er problemer føler kunden at deres stemme bliver hørt. Dette giver kun mening for dem da der ikke er mange problemer med deres produkt. Hvis der var problemer ofte, ville de ikke lavet andet, men det er ikke ofte at der kommer noget frem.

Det viser også at kunder der kommer ind ad døren, er glade for at være der, men en kunde som ikke er inde endnu, har ikke et så godt overblik over hvad de lige-præcis får for deres penge. Det er blandt andet derfor de oftest går efter de største kæder først, da de kan bruge meget tid på at få dem overtalt. Og det er der hvor problemformuleringen for denne opgave kommer ind.

Det giver også nogle gange mere mening for større kæder at bruge GoFact, en mindre kæder, da de også har nogle features som hjælper mere hvis man har flere butikker der snakker sammen. En af dem er en "bytte forslag" system. Det kigger kort sagt på et produkt, og hvor det bliver solgt mest henne. Det kunne være en striktrøje som sælger mere inde i Herning end i Brande. I stedet for at butikken i Herning køber flere nye trøjer, kan de tage nogle af dem butikken i Brande sidder med. Dette gør at de forskellige butikker ikke har spild af det tøj de ikke kunne få solgt. Det er her at hvis man er en lille kunde ville man ikke have andre butikker at kunne bytte med. Det er også en af grundene til at det ikke er en af de features fra GoFact som kommer med i vores light version, da det er bygget til de små, nye kunder.

GoFact fungerer også som en data-backup, da kunder skal sende en kopi af deres data til dem, har kunderne det hos GoFact, og GoFact har flere backups hvis der går noget galt. Så man kan også være sikker på at intet data går tabt når man bruger dem. Det sparer også kunderne for at kigge på en decideret backup løsning. Det gør GoFact til et system der håndterer kundernes data, men også sikrer det.

Problemdomæne

Når man gerne vil have sit produkt ud i verden, kræver det en god strategi. En stor del af ens strategi er grundet i hvilket produkt man laver. Hvis man laver en køkkensvamp, er ens målgruppe en gennemsnitlig person. Men den kunne også være et industrielt køkken. Man kan dermed sige at man kan sælge få svampe til mange mennesker, eller mange svampe til få institutioner. Dette er det samme problem som GoFact også har siddet med. De har et software som er godt til at sænke spild, og øge vækst for tøjbutikker. Men hvem skal man fokusere på? Hvis man kunne tilkoble alt og alle til systemet ville man kunne gøre begge nemt, men det er ikke sådan det virker for GoFact, på grund af deres ”kom som i er” mentalitet. Det gør det nemmere for nye kunder at tilkoble GoFact fra kundernes side, men for GoFact skal de vælge hvilke kunder de tager ind med lidt omhu, da det tager GoFact en del mandtimer for at få tilkoblet en ny kunde. Det er også der deres største udgift ligger, da deres abonnement model gør at de ikke skal tænke mere på en kunde efter de er opsat.

Det gør det svært for GoFact at få nye kunder. De har et godt produkt, som de har bevis på at de sparer butikker mange penge hvert måned, og mindsker tøj-spild. Der er også mange butikker som gerne vil blive en del af GoFact systemet, men da det ikke er alle der ville kunne få lige så stor værdi af det, kan det være at nogle går fra efter det hele er opsat. Det er et stort problem for GoFact hvis en kunde går fra, da de i opstarten af en ny kunde laver meget ”gratis” arbejde, i det at de ikke tager penge for opsætning, bare den månedlige pris. Det betyder at hvis en kunde tager 250 timer at få sat op, og de går af efter første måned, kunne GoFact tabe mange penge, da de tjener tabet op i de følgende måneder.

Kunderne kan nemlig gå fra for flere årsager. Det kunne være at de ikke ville bruge systemet alligevel hvis det bare ikke går nemt ind i deres IT Pipeline. Hvis de ikke kan se en tydelig værdi i de første måneder. Hvis deres budget ikke har plads til en ekstra IT-system. Eller hvis de skærer ned i mængden af eksterne systemer.

Den sidste lyder ikke så mulig, men for nylig havde GoFact en stor kunde som de var i gang med at få ind i deres system. De havde snakket frem og tilbage med forskellige personer og afdelinger hos denne store kunde, i over et års tid. Det havde været et kæmpe tab for GoFact, men hvis de kunne få det hele til at virke, havde de fået en massiv kunde ind, og ville have tjent mange penge hver måned som resultat. I dette års tid havde der været skrevet mange linjer kode. Skrevet mange custom løsninger til lige netop denne kunde. Men de endte med at rydde op i deres eksterne IT-systemer, og dermed var alt det arbejde fra GoFacts’ side ligegyldigt.

Det var en kæmpe sats, som ikke betalte sig alligevel. Og det er her deres nuværende business strategi fejler. Hvis de kan få deres store kunder ind ad døren, er det en kæmpe succes. Men hvad hvis de ikke kan det? Hvad hvis kunden går fra for ingen skyld af GoFact? Deres strategi gør at hver kunde er meget vigtige for dem, og dermed kan hver eneste kunde der går fra være et kæmpe tab for dem. Man kan sige at de er meget følsomme for ændringer i markedet, og det kan ses som et problem.

Så hvad kan et fix til dette problem være? Få flere kunder ind. Få mange, mindre, kunder ind ad døren, så den kollektive mængde kunne gøre tab mindre vigtige. Det ville give en sikkerhed hvis en stor kunde går fra, da de stadig har mange forskellige kunder som ikke ville gå fra dem på samme tid. Der kunne stadig være mange kunder der går fra, hvis en ekstern ekstrem handling sker (som en anden Corona virus, ect.). Men det ville give en sikkerhed i normale tider som de ikke pt har.

Men så kommer vi tilbage til deres hovedproblem. Hvordan får man flere kunder ind, når man allerede bruger alle mænd til at få kunder ind? Man kigger på hvor tiden går hen. På GoFact er hoveddelen af tiden brugt på at skrive custom pipelines til de individuelle kunder. Men en stor del af tiden går også til at evaluere, og snakke med potentielle kunder, for at se hvem der kunne have størst chance for at blive efter. Hvis man kunne sænke denne tid, kunne man optimere hvor mange kunder man kan få ind ad døren på et år.

Med mindre butikker som ikke har for fastlagte IT-systemer kunne man også sige at de skal ændre deres data til en "simplere" version af GoFact systemet, som ikke er custom lavet til dem. Man kunne give denne version en billigere pris, da GoFact ikke skal gøre noget for at få en ekstra kunde ind ad døren. Det ville også være mere attraktivt til små kunder at det er billigere for dem.

Dette er dog ikke hvad der er fokus på her. Vi fokusere på at sælge ideen til kunder. Vi skal dermed komme frem til et produkt der nemt kan vise hvad GoFact kan gøre for kunder. Det skal kunne hurtigt highlighte i en praktisk måde, hvad GoFact kan gøre for kunderne. For det kan vi kigge på hvad vores produkt skal kunne gøre, hvordan vi ville udføre dem, og hvad vi vælger at skære fra.

Det er også vigtigt at vi skærer funktioner fra, da vi prøver at sælge hoved ideen på GoFact. Det er en bedre salgs-pitch at komme med en god grund til at bruge en, i stedet for mange små. Da folk nemmere kan have små vigtige ting i hovedet, end mange små, mindre vigtige ting.

Hvordan har vi tænkt os at vælge hvad produktet skal kunne?

Når man vælger hvad et produkt skal kunne, og hvad den ikke skal kunne, kan man bruge værktøjer til at hjælpe med dette. Det værktøj jeg har tænkt at bruge her hedder MoSCoW modellen. Den simplificerer hvad et produkt skal have for at det er en MVP (Minimal Viable Produkt) og hvad der gør den mere fuldendt. Ved at bruge MoSCoW modellen kan man også se nemt hvad man fravælger, og hvad man optimistisk håber man kan få i sit produkt inden en deadline.

Den simple MoSCoW model uden forklaringer kan findes i Bilag.

MoSCoW Modellen

Must Have

For at sige at vi har en light version af GoFacts' system, skal vi have hoveddelen på plads. Programmet skal kunne tælle salg sammen, sortere data på en måde der gør det nemt for kunden at se hvad der har solgt. For at komme dertil har vi følgende punkt-liste af features programmet skal kunne.

Nr. 1 Loade en kundes fil nemt

Når vi snakker om onboarding af nye kunder, skal vi gøre det så nemt for dem så muligt at se vores vision, og hvad vi kan gøre for dem. Selvom vi snakker om en lightweight version kun skabt til at sælge en vision, er det stadig vigtigt for os at det er så nemt som muligt. Vores produkt har funktion at sælge en vision. Og hvis vores salgs pitch er svær at bruge, reflektere det ikke godt på den fulde version. Det er dermed vigtigt at vores version er ligeså nem, hvis ikke nemmere at bruge end det fulde produkt. Men hvad er "nemt" at bruge? Det fulde system kører data automatisk ind, så hvis vores "nemhed" kriterie går imod manuel vs. Automatisk, kommer vi til at fejle på dette punkt uanset hvad. Så hvad kan vi gøre for at det er nemt? Genbruge systemer kunderne allerede kender til. Vores input til vores system skal bruge indbyggede filnavigation, og ikke noget custom lavet. Så i stedet for at have en GUI eller en anden løsning som man kunne blive forvirret over, skal vi ligge en fil ind i en mappe tydeligt navngivet "input". Det ville være det nemmeste at fortælle en kunde. "læg den eller de ønskede filer ind i denne mappe og kør programmet". Mappen bliver lavet på ens skrivebord, som er der hvor de fleste mennesker opbevarer deres "vigtige" applikationer. Så vores input bliver så tydeligt og nemt som muligt.

Nr. 2 Lave udregninger på data

Når vi har data inde i vores system, skal vi også gøre noget ved det. GoFact laver mange forskellige udregninger på deres data. Men da vi bare skal sælge vores vision for hvad vores software kunne bruges til, kan vi holde os til nogle af de grundfunktioner som gør GoFact specielle.

Det mest essentielle er at fortælle en hvor mange salg hver genstand har haft. Dette gør det nemt for en butik at se hvad der sælger, og hvad der ikke gør. Det vigtigste step er dermed en optælling af kundens rå data. For at gøre det, skal vi vide hvordan sådan en fil ser ud. De indeholder en identifikator til et produkt. Det kaldes en EAN-kode. Der er også hvor mange produkter der var i salget, samt for hvor meget det hele kostede. Det er de vigtigste data for os, men vi kommer også til at mangle noget mere da vi ikke har et kæmpe katalog af EAN-koder og de produkter de peger hen til.

For at få den samme funktionalitet ud af vores program som GoFact, skal vi tilføje nogle felter som nogle gange er der, og andre gange ikke. (igen snakker vi om systemer som ikke er standardiseret) De felter vi tilføjer som oftest ikke er med i disse typer filer er "CostPrice". Altså hvor meget kostede varen. Vi tilføjer også en "SalesPrice" felt, som er hvad en af disse produkter sælges for. Det sidste felt vi tilføjer er et "gross profit" felt. Det er den totale mængde profit der kom af salget. Det er dette man kigger mest på når man ser hvilke produkter man skal fokusere på, men man kigger også på antal solgte, da det er de varer folk køber mest, er der mest potentielle på dem.

Nr. 3 Sortere efter industri standart og højeste salg

Vi skal vise data på en måde som gør det nemt for kunden at se hvad der er solgt, og hvor meget. Oftest sortere man efter størrelse indenfor hvert produkt. Men vi skal også have højeste salg først, da det er dem vi skal vise til kunden.

Vi ender med en mærkelig blanding af industri standarten, og min egen version, da jeg ikke har et katalog af produkter, bliver hver EAN kode et helt unikt produkt, hvor man oftest ville have en række af EAN koder der er "samme" produkt, men en anden størrelse eller anden farve. Det betyder at i mit program bliver der sorteret efter *alle* størrelser, og indenfor samme størrelse, antal salg. Det er ikke helt som man gør i den virkelige verden, men for at få det til at passe til den virkelige verden ville jeg skulle have rigtig data, i stedet for data jeg selv laver.

Normalt har man også mulighed for at vælge om man vil kigge på det første eller det andet tal, i størrelser der har 2 værdier. Det kunne være BH størrelser, som har både omkreds og kop størrelse, eller bukser som har længde og bredde. Det burde ikke være så svært at implementere, så for at komme tættere på hvad man ren faktisk har i industrien, vil jeg også have denne feature med.

Nr. 4 Vise resultatet nemt

For at sælge firmaet til en kunde, skal det være nemt for kunden at forstå hvad de kigger på. De fleste mennesker har erfaring med Excel, og det er også ofte det salgs afdelinger bruger. Derfor skal vi kunne udskrive vores resultat i Excel. Det ville være det nemmeste at forstå for en gennemsnitlig person, men hvis vi snakker med kundes IT-afdeling, kan vi udskrive det i JSON. Det er også et meget kendt format, men det kræver oftest at man har med noget IT at gøre for at man har set det før. Så hvis man skulle overbevise IT afdelingen hos en kunde ville det være smart med denne mulighed.

Vi beholder også den originale fil, da det kunne være data kunden ikke vil miste. Derfor skriver vi en ny fil med output, og putter det ind i en anden mappe så kunden ikke bliver forvirret over hvilken fil er ny.

Should Have

Nu er vi i "should have" og her handler det om "Hvad skal vores program have, for at det er lidt mere end bare minimum". Jeg vil gerne have disse funktioner i programmet, så det er mere end bare det mindste der ville være acceptabelt.

Nr. 1 Syntetisk data

Jeg kan desværre ikke bruge en rigtig fil, da kunderne hos GoFact forventer at ingen ser deres data. Men jeg kan lave mine egne, som indeholder noget ens, men ikke det helt samme, som de rigtige filer. Dette er blandt andet for at være sikker på at jeg ikke giver noget væk jeg ikke må, men også for at fjerne unødvendige felter som kun ville tilføje mere manuelt arbejde, men ingen ekstra funktionalitet.

Det handler ikke om personfølsomme informationer, men det er en brud på GoFacts' regler. Det betyder at jeg ikke kan bruge en rigtig fil, men jeg kan tage inspiration fra dem til at skrive min egne.

For at teste forskellige kombinationer, og vise at det bliver gjort live foran kunden, skriver vi noget kode som *automatisk laver forskellige produkter*. Det skal indeholde en tilfældig blanding af EAN-koder, Størrelser og priser. Det skal ikke være helt tilfældigt, da vi skal have mulighed for at sælge den samme vare flere gange. For at få begge disse krav laver vi en liste over produkter, og når vi "laver et salg" i vores fil, vælger vi en af disse forskellige produkter. Det betyder at vi laver nye produkter en gang ved hver gang vi kører programmet, og derefter genbruger dem vi har lavet.

På denne måde har vi de felter vi skal bruge for at programmet kører, uden at bryde nogle Terms of Service, og vi har noget kunden kan se hvis de ikke gider at bruge deres egne data.

Nr. 2 Support til flere filer på en gang

Programmet vil ikke have noget dato felt, da det kan være at kunden ikke har filen adskilt efter dato, og vi ville samle data som ikke skulle samles. Så hvis en kunde har en ny fil per måned, eller hvad nu end de har, skal vi kunne køre flere filer på en gang. Dette er for at gøre det nemmere at vise programmet.

Nr. 3 Fejl håndtering

Når der er en fejl i GoFacts' system, bliver det sagt videre til kunden så de kan fixe det. Der er selvfølgelig fejl-beskeder inde i GoFacts' system internt, men hvis det er noget kunden, har brug for at vide bliver det givet til dem. Vores program skal også kunne forklare simpelt hvad der er galt. Et eksempel på noget der kunne have været gået galt, er hvis den delimiter der bliver brugt i filen er det samme som decimal tegnet. Det ville resultere i flere rækker i dataen end der er i headeren. Dette ville ødelægge programmet, men potentielt ikke på en måde der ville crashe systemet. Det ville betyde at en outputfil ville stadig blive lavet, men det ville være forkert. Hvis vi viser det til en kunde uden de kan se hvad der er gået galt, tænker de at vi ikke har et produkt der virker. Derfor skal vi have tjeks til at se om noget i denne stil sker, så vi kan fixe det foran kunden og prøve igen, uden at de tænker at vores system ikke virker.

Nr. 4 Unit tests

Unit tests i softwareudvikling er meget vigtige. De bidrager ikke til funktionen af et program, men gør det nemt og hurtigt at teste hvis man har mange dele der hænger sammen. Mit program ville køre ret sekventielt, og jeg ville splitte ansvar op så det er tydeligt at se hvor noget går galt. Dette gør det nemmere at se fejl manuelt, men Unit Tests ville stadig være godt til at sikre sig at alt kører ordentligt. Der er ikke så mange grunde til ikke at have Unit Tests, andet end opsætningstiden. Men det burde ikke være det største problem, og jeg har dermed også tænkt mig at have dem.

Could have

Når vi snakker om could have, snakker vi om ideer. Jo, Must have og Should have er også ideer, men de skal være der for at programmet er en succes. Men could have er mere frit. Hvis vi havde uendeligt tid og ressourcer kunne vi lave alt på listen, men disse ting er mindre vigtige end resten. Hvis de ikke når at blive lavet gør det ikke noget, da vi stadig har det program vi ville have.

Nr. 1 Flere farver i konsollen til at øge læsbarhed

De fleste mennesker er ikke vant til konsol applikationer. Derfor skal vi prøve at gøre det så nemt for dem som muligt

Nr. 2 En hurtigere måde at ændre filen på en manuel arbejde

Måden vi har vores program til at være hurtigere end GoFacts' system er at vi går ud fra at vi ved hvad der er i headerne. Måden dette skulle ske var at vi manuelt tjekkede den data der var i filen før vi kørte den igennem programmet. Men man kunne have en konsol interface hvor man kunne angive header navne, eller hvilken række nummer som den værdi havde. Dette er et blødere krav, og betyder ikke at programmet skal have det. Men det ville være en rar feature hvis den havde.

Wont have

Wont have er sektionen hvor vi afskærer features fra programmet. Det er ting som man kunne have med, hvis man havde ingen grænse på tid eller ressourcer. Det er også her hvor man kan fjerne features som ikke ville bidrage til programmet men tager tid at implementere, eller i værste tilfælde trække fokus fra vigtige områder.

1. GUI
2. Advanced GoFact Features

Nr. 1 Graphical user interface

Noget som man i første tilfælde kunne tænke ville være en god ting at have med i sit program, ville være en grafisk brugergrænseflade. Det er ofte hvad man er vandt til når man bruger et program, og hvis mit mål er at gøre det så nemt for brugeren som muligt, hvorfor vælger jeg så ikke at have det med i mit program?

Simplicitet. Der er mange problemer man kommer ind i når man har en GUI, og det er heller ikke alle applikationer der kræver en. Mit programs input er filer i en folder. Det er ikke komplekst eller kræver andet input, og dermed kunne man spørge ”Hvad skulle en GUI i dette program vise?”

Status over hvor langt den er i processen? Programmet er meget hurtigt, og når næsten altid at være færdig så snart man trykker på start.

Input? Hvorfor have input på en GUI man også skal lære, hvis man allerede bruger filsystemer ofte?

Mit program kræver ikke en GUI, da den ikke ville bruge den. Så hvorfor involvere grafisk design, testing af UI og en masse programmering, for en feature som ikke ville bidrage til funktionen af produktet? Det er derfor jeg har valgt ikke at have en GUI til mit program. Programmet er simpelt, og det ville kun forvirrer med at have en GUI.

Nr. 2 Advanced GoFact features

GoFact har som sagt mange flere features, som alle forbedrer oplevelsen af at bruge deres produkt. Tidligere nævnt var der ting som "Hot-meter" som siger hvor stort potentielle en varer har, baseret på hvor mange der har solgt på en given tid. Men da jeg selv har set "bag scenen" og kender til hvad der går ind i sådan en udregning, ved jeg at der er mange flere faktorer der spiller ind i den udregning. Det er simpelt at sige hvad den står for, men der er også faktorer som kræver hele deres system for at kunne give en præcis værdi. Det er noget de selv har opfundet, og har dermed ikke en offentlig udregning de bruger til at regne den ud, da det er deres IP.

For at lave noget ens til den udregning kræver det også at jeg har en lager-fil som indeholder hvor langt tid noget har været på lager, som jeg ikke har i mit lightweight version, da fokus er på en kundes salgs fil. Jeg kunne tilføje det til mine syntetiske data for at "fake" en "hot-meter". Men det ville så være noget data som jeg ikke kunne give kunden hvis de ville køre deres egne filer igennem, og det ville være svært at vide hvad det betyder, da det ville være tilfældige tal bag og ville dermed ikke give nogen mening.

Der er også andre features som GoFact har, så som en AI ChatBot som kan læse alle sine data, forstå dem og give anbefalinger ud fra dem. Det er features som mange udviklere har arbejdet på i flere måneder, og er langt ude af Scope af denne lightweight version.

Vi kan dermed sige at vi fokuserer på de dele som er vigtige til at sælge GoFact som ide, uden at forvirre kunden med en masse features som de ikke ved om de vil have endnu.

Metode Valg

Fil struktur

Siden dette program er en GoFact light, så ville jeg gerne selv holde det så tæt på GoFacts' fil struktur som muligt. Tanken var lidt at jeg syntes det gav mening, men også hvis nogen fra GoFact skulle arbejde videre med dette projekt, så kunne de nemmere forstå hvordan programmet fungerer.

Så hvordan er GoFacts' system bygget op? De bruger forskellige pipelines til hver funktion. Så man kunne næsten sige at de har mange forskellige programmer kørende. Så når data kommer ind, kommer det igennem en masse forskellige scripts. Dataen bliver rensset, og bliver lavet om til noget deres system kan forstå. Det loop (med flere steps) bliver kørt igen og igen for hver fil, og deres resultat bliver lagt klar til næste trin i processen i en buffer de har liggende i skyen.

Tanken med denne tilgangsmåde er at hvert trin er isoleret, og hvis noget går galt i en af dem, så er deres resultater den har lavet indtil da ikke gået tabt. Plus, siden de forskellige dele af deres kode kører hurtigere end andre dele, betyder det at de ikke har en kæmpe flaskehals. Så når deres "Translator" (Den der oversætter kundens data til data GoFact kan bruge) er færdig med en fil, skal den ikke vente til alle andre steps nede af kæden skal være færdige med sit arbejde, før den kan gå videre til den næste fil.

Det lyder meget godt, men hvordan kan jeg gøre noget lignende i mit program? Det er meningen at jeg skal lave en lightweight version til en kunde, så have pipeline buffers i cloud kommer til at være meget ekstra for ingen grund. Og da vi ikke laver nogle alt for store udregninger, er hastighed ikke det største problem. Så en buffer mellem de forskellige arbejdsområder ville ikke give meget værdi. Men vi kan stadig "emulere" deres struktur.

Praktisk

Jeg har tænkt mig at bruge deres pipeline struktur. Måden de har deres filer opdelt på, er at de har navngivet hvert trin med et tal, og så har de under mapper og scripts inden for dens opgave. Så når jeg tænker på min proces, så laver jeg følgende mapper. "0.Common". Jeg tænker at starte med 0 da programmering oftest er nul-indexeret. Jeg valgte også at bruge nul til "common" filer mellem alle dele. Det er ting som min DBO til en CSV-fil, men også ting som min "OutputFormatter". Tanken er at de ting som flere steder skal tilgå, kan alle komme i samme sted. Så vedligeholder jeg strukturen af at daten kører igennem projektet i en blød smøre. Og ikke springer frem og tilbage mellem filerne.

Det første trin i programmet er at indlæse filer fra en CSV fil. Grunden til at input er en CSV fil er at de fleste mennesker bruger den filtype. Da Excel kører på en filtype der er tæt på, kan man oftest lave en Excel fil om til CSV, og man kan åbne en CSV fil i Excel uden nogle problemer. Jeg vil kalde dette trin for "1. Loading Files". Så alt der har med indlæsning af filer at gøre kommer i denne mappe. Efter filer er indlæst, er dette sektionens opgave færdig.

Næste trin i processen er "2. DataHandling". Det er alt der ændre på dataen der kommer her ind. I det fulde system laver de meget i dette trin, men vi holder os til 2 primære opgaver. Komprimere data, og sortere data. Komprimere data handler om at samle mange af samme slags salg til en, hvor den samler hvor mange salg der er lavet på det objekt, samt hvor mange penge man har tjent på salgene.

Den anden opgave DataHandling trinnet har, er at sortere data. For at vise data til kunden i den industri standart de forventer, sortere vi per kategori, størrelse og dernæst antal salg inden for samme størrelse. Resultatet af dette trin ville være en sorteret liste af samlede data. Dette er helt klart det største trin, og er det der giver det mest komplekse kode, men det kigger vi nærmere på lidt senere. Herefter er dataen klar til at blive præsenteret til kunden.

Det går ind til det sidste step i vores filstruktur, "3.WritingOutput". I dette trin handler det om at skrive data til kunden så de kan se resultatet af vores program. Dette inkludere både til konsollen med opdateringer om programmets handlinger, og til filer som kunden kan læse som viser resultatet. Det med at skrive til konsollen bryder lidt den tanke om at have fælles ting i common, men da den skriver information ud, giver det ikke mening at opbevare koden et andet sted end i mappen der har jobbet at udskrive data.

Konsollen skal have flere farver til at signalere forskellige koncepter til kunden. Dette er vigtigt da de fleste mennesker ikke er vant til at læse konsol output. Flere farver til forskellige informationer ville hjælpe kunden til at forstå hvad der sker. Jeg har tænkt mig at adskille "fyld-tekst" fra blandt andet fil-stier, mappe-stier, fejl beskeder og succes beskeder. Hver af disse skal have en anden farve. Fyld-tekst bliver hvid, fil stier bliver gule, mappe stier bliver orange, fejl bliver røde, og succes bliver grøn.

Og når vi skal vise output af programmet, skriver vi til både CSV, og til JSON. Tanken er at hvis vi snakker med IT afdelingen kan vi bruge JSON, da det er noget de kender mere til, og hvis vi snakker med en salgs-person eller butik ejer kan vi bruge CSV. For ikke at have noget ekstra input der forvirrer i starten, skriver vi bare ud til begge filtyper alle gange. Man kan altid ignorere den anden fil der bliver lavet. Det viser også kunden at der er muligheder til at give dem data på en måde de kan forstå, samt viser det at processen er nem at starte med.

Databærende objekter

I min filstruktur har jeg gjort plads til forskellige objekter som kommer til at gå gennem mit program. Jeg har tænkt mig at ligge alle former for DBO'er (Databærende objekter) i denne fælles mappe. Det første jeg tænkte på at bruge DBO'er til var at opbevare dataen fra filerne i C# hukommelse. Det ville også gøre det nemt for mig at tilgå data, så jeg ikke hele tiden skulle gå tilbage til filen og læse hvert felt ind hele tiden. Jeg tænkte at have en DBO som bare indeholder en liste af strings. Denne DBO ville være til for at opbevare den rå data fra en fil. Så vi skriver alle data typer til Strings først, og så dernæst omskriver dem til de rigtige datatyper. Denne DBO ville blive kaldet for "Rows". Den indeholder data for hver row, men så har jeg også brug for en anden DBO til at have en liste over alle Rows i en fil. Den vil jeg kalde "CSV_DBO", da den indeholder data om en hel CSV-fil.

Det var de rå data, men jeg skal også have dataen i de rigtige data-typer, så jeg vil også have en lignende DBO-struktur der indeholder det. Jeg har tænkt mig at tage lidt inspiration fra GoFact, og bruge et navn de havde der hedder "CanonicalModel". Det er også et design pattern, som er til for at hjælpe med dataoverførsel mellem forskellige filsystemer. Tanken er at man har et DBO der indeholder felter til alle de forskellige data navne og typer man skal bruge. Så skal man ikke have mange forskellige med de forskellige navne og typer af data.

Jeg har så tænkt mig at have min CanonicalModel indeholde alt data fra en CSV-fil i de rigtige typer. Den indeholder så data fra en row, og jeg skal også have en DBO til at indeholde alle de CanonicalModels. Jeg har valgt at bruge en anden term fra GoFact, som jeg ikke bruger korrekt, men jeg kunne godt lide navnet. "InhouseData" bliver normalt brugt til at referencer til det data et firma laver. Det er ikke helt det jeg bruger det til, men GoFact laver deres CM's (CanonicalModels) om til InhouseData, så jeg prøver at holde den samme struktur som de gør.

Hver instans af InhouseData indeholder så alt oversat data fra en CSV-fil, da den indeholder en liste af CM's. Siden jeg skal kunne håndtere flere filer på en gang, kan jeg have en liste af InhouseData og sende den frem og tilbage mellem de forskellige trin.

Komprest data

Sådan en salgsfil som vi skal loade ind, indeholder meget af det samme data. Det samme produkt kan sælges flere gange på en måned, eller hvor lang periode der nu er givet i filen. For hvert salg kommer der til at være en ny linje hvor der står hvilket produkt der er solgt, og hvor mange der er solgt osv. For at vi kan vise et resume over alle salg, skal vi på en eller anden måde tælle alle salg af samme produkt sammen. Når vi tæller dem sammen, skal vi have en sum over hvor mange der er solgt, så kan vi gange op hvor mange salg der er lavet med profit på varen for at se hvor mange penge man har tjent på den varer. Det data der bliver samlet her er hvad der ender med at blive udskrevet til kunden, så vi kan tilføje ekstra detaljer som vi kommer til at skulle bruge senere i programmet. Nogle af disse detaljer kunne være hvor meget der er solgt indenfor en kategori, da vi kan udskrive et resume af resuméet. Så vi viser både et total mængde profit, men også en mere fuldendt version hvor man kan se hvilke produkter der har solgt mere end andre.

Sorterings Metode

Målet er at samle or sortere data, men det er ret komplekst problem. Det at sortere er et løst problem indenfor kode-verden, med mange forskellige måder at sortere på. Jeg har dog tænkt mig at bruge "Insertion sort". Det er nok den mest simple løsning, men det betyder ikke at den er dårlig. Insertion sort kigger ret simpelt på om et element er større eller mindre end den før. Hvis den er mindre, vil den bytte om på de to elementer. Det virker meget godt for små datamængder, men da den skal gå igennem hvert element, måske flere gange har den nogle skaleringsproblemer. Når jeg tænker på de salgs filer jeg har set er de ret lange, men indeholder oftest ikke alt for mange produkter. Vi snakker om måske 60 produkter i gennemsnit, og det er intet problem for Insertion sort.

Hvad gør det så komplekst? Vi sorterer jo ikke bare på tal. Hvis det er skostørrelser vi sortere i er det meget nemt. Vi sorterer bare tallene som normalt, men hvad hvis vi sortere BH-størrelser? Hvad hvis vi skal sortere small medium og large? Det er her det bliver meget komplekst. For hvordan ved man hvad man sorterer?

Så for at sortere de forskellige tøjstørrelser kommer jeg til at skulle bruge en smart sorterings algoritme. For at kunne sortere disse tøjstørrelser kunne jeg prøve at oversætte en størrelse som ikke er i tal, til et der er i tal form. Hvis jeg kan få det til at virke ville jeg kunne bruge den simple sorterings algoritme i stedet for at skrive en custom sorterings algoritme som skal kunne kende forskel på bogstaver og tal, og vide hvad de betyder. Så man kan sige at jeg "oversætter" dataen til et andet format.

Det kommer til at tage en del tid at implementere, men det er vigtigt at give data til kunden på en måde de kan forstå. Jeg ville ikke kunne sige at jeg har nået i mål hvis kunden ikke kan finde rundt i de forskellige produkter. Det er også noget GoFact gør i deres hoved-system, så for at passe deres hovedfunktioner kræver det også at jeg gør dette trin. Jeg forventer at bruge noget Regex til at identificere de forskellige størrelser, samt til at få dele af størrelserne for sig selv. Så som bogstavet i BH-størrelser. Så ville jeg kunne tilføje kopstørrelsen til omkreds størrelsen for at få et tal der bliver sorteret på. Men for at de bliver sorteret efter kopstørrelse først, og omkreds anden, kunne jeg gange et stort tal til kopstørrelsen så det betyder meget mere end omkreds. Så størrelsen "A50" ville blive til noget lignende "1050" hvor de 1000 kommer fra kopstørrelsen, og de 50 kommer fra omkredsen.

Til T-shirt størrelser med XL og 2XS osv. Har jeg tænkt mig at splitte M S og L, og så se på hvad der er tilbage. Jeg kan ikke bruge deres Char værdi nummer, da de er i omvendt rækkefølge af hvad de ville skulle være. Altså L er mindre end M og S. Så jeg skal bruge noget custom til at lave værdierne om til 1 2 eller 3, og så gange det store tal på det. Hvis der så er 2X tilbage efter jeg har fjernet L, kan jeg tilføje de ekstra X'er til det store tal vi har lavet før.

Teori

Ikke alle teknologier jeg har brugt er noget der er intuitivt, eller som de fleste programmører ville vide. Primært er der snakke om Regex, men også en custom hashing algorime, som omdanner tekst til tal. Der er ikke noget specielt med den algoritme, men tankegangen er ret unik, og lidt svær at forstå bare ved at se koden, så vi forklarer det også her.

Regex

Regex er noget nogle programmører har hørt om. Men det er noget som de færreste ren faktisk forstår. Da det er en stor del af mit program har jeg skulle blive en ekspert på området. Det har indebåret at lære syntax, læse massere af Regex, og lære at skrive dem selv. Samt tænke på hvad Regex Engine gør, og hvordan man optimere dem.

Min brug af regex opsummeret

Men hvorfor skrive dem selv, i stedet for at bruge AI til at skrive dem for en? Hvis man kun skal bruge simple ting, kunne man sagtens bruge AI til at lave ens Regex Expressions, men når det bliver komplekst kan det være svært at skrive alle de forskellige cases som den skal indebære, i prompts. Et eksempel på en kompleks regex kunne være denne i figur 1. Jeg farver hver Regex med en gul baggrund. Det viser hvad der er regex fra koden, og hvad er tekst der skal læses.

```
"^([0-9]*[A-z0-9]*([0-9]*)([0-9]*?)([0-9]*)([0-9]*?)([0-9]*)([0-9]*?)([0-9]*)[0-9]*$"
```

Figur 1 - Universal Regex

Den er meget lang, og indeholder mange af de samme elementer, men når der står 0-9 kan det betyde flere ting, angående hvad der er før eller efter. En AI kunne måske skrive denne regex, men den bruges meget i mit program, hvor jeg kalder på forskellige dele af den flere gange, så det ville ikke bare være at bruge AI til at lave sådan en Regex. Men det ville også kræve AI hver gang man skulle bruge den, eller hver gang man skulle debugge den. Det fact har gjort at jeg har valgt at skrive mine Regex manuelt for at sikre at jeg ved hvordan jeg bruger dem, samt at jeg kan selv vælge hvor optimeret de er, og hvor læsbare de er. Tilgangen krævede dog at jeg at lærte Regex fra bunden af.

Men hvordan ser det ud i det større sammenhæng? Mit program skal være brugbart for dem hos GoFact, og det betyder at andre programmører skal kunne læse og forstå min regex. For at sørge for at det er nemmere for andre at læse min Regex, har jeg prøvet at gøre den mere "begynder" venlig, uden at komprimere på performance. Meget af den regex kunne have været "simplificeret" ved at bruge `\d \D` og `\W`. Men det er forkortelser for hvad regex søger på. Hvis man ikke ved hvad de står for kan det føles umuligt at forstå. Forskellen på `\d` og `\D` er også kæmpe (bogstaveligt talt modsat), men de kan se ens ud til nogen der ikke ved det. Jeg har dermed valgt at bruge hvad de symboler står for, i stedet for at bruge symbolerne selv. Jeg ved at de bruger Regex ude hos GoFact, men jeg ved også at ingen af dem kan lide at bruge det, og ved bare det minimum de skal for at kunne bruge det. Det er blandt andet også derfor at jeg har valgt at skrive den med `[0-9]` i stedet for `[\d]`.

Optimering af regex

Når man bruger regex, kan det enden være meget hurtigt, eller meget langsomt. Det kan føles lidt som en 50/50 en gang imellem, hvis man ikke forstår hvad der sker. For mit brug af Regex, kræver det at jeg optimere dem så meget som muligt.

Samle flere regex til en

Når man snakker om regex optimering handler det primært om 2 ting. At sørge for at man ikke kører mange små regex queries, og sænke backtracking. Grunden til at man samler flere regex i en, er at der er noget overhead i at starte Regex Engine. (Regex Engine er hvad der kører regex bag scenen). Hver gang man kalder en `.Match()` eller `.IsMatch()` på regex kræver det at ens computer allokere hukommelse, og den laver en del funktion kald bag scenen. Dette er oftest ikke et stort problem hvis man kalder dem få gange, men hvis man kan optimere 100 kald ned til 10 eller 1, ville man se en stor forskel på længden.

En måde man kan sørge for at man ikke skal køre mange forskellige regex kald på, er ved at samle flere expressions til 1, ved hjælp af `|` symbolet. Den gør det samme som i programmering, og betyder "or". Så man siger "passer mønster 1, eller mønster 2". Man skulle tænke at det ville være det samme, hvor den kører mønster 1 igennem først, og dernæst mønster 2. Men i virkeligheden tjekker den begge mønstre igennem på første gang.

Sænke backtracking

Meget af tiden Regex bruger til at udfører en søgning, er ved at lave hvad man kalder for "backtracking", som betyder at den løb ned af den "blind gyde" og skulle gå lidt tilbage igen for at kunne fortsætte. Disse kan ske meget ofte hvis man ikke passer på, og minimere disse er den anden primære optimerings metode man har i regex.

Den nemmeste måde at minimere backtracking, er ved at være mere specifik i sin søgning. Et eksempel på dette ville være hvis man søger på bogstavet, "E" i en tekst. Den skal undersøge hver eneste karakter i teksten for at finde bogstavet E. Men hvis man kun vil se om et ord starter med et "E", kan man tilføje det kriterie der hedder "\s". \s er "whitespace" karakter. Det at tilføje det kriterie gør at den ikke længere ser efter hvert eneste bogstav, men at den nu kun leder efter bogstavet E i starten af et ord.

Et andet eksempel på at være mere specifik, er at have høj "kontrast". Tanken er at man sørger for at Engine skal gætte så lidt som muligt. Den gætter oftest når man ikke er specifik nok. En nem måde at øge kontrast på, er ved at undgå "." karakteren. Den matcher alt, og dermed matcher den alt den kan, og dernæst går tilbage for at finde stedet hvor resten af mønstret passer. Så i min opgave, i stedet for at bruge "." til at finde mellemrum, eller special tegn som / og - kan jeg sige \d og \D. Dette finder alt der er tal, og så alt der ikke er tal. Tanken er at grænsen mellem dem er så hård som mulig, og dermed ved regex med det samme hvilke dele af teksten hører til mønstret.

En regex i koden forklaret

Jeg viste tidligere en Regex fra min kode i figur 1, men det ville være bedre at forklare en anden, da den i figur 1 har et meget bredt mål. Har denne i figur 2 et simplere, nemmere at forstå mål.

```
^([sS])$|^([mM])$|^([lL])$|^[Xx]+([Ss])$|^[Xx]+([Ll])$|^[0-9]{1,2}[xX]([Ss])$|^[0-9]{1,2}[xX]([Ll])$
```

Figur 2 - TShirt Regex

Dens mål er at finde ud af om den tekst vi tjekker igennem, starter med en T-shirt størrelse. (2XS, M, L, XXXXXL osv.) Samt finde ud af hvad det er for en. Den ser måske meget kompleks ud, men det er blandt andet på grund af princippet om at samle flere regex i en. Hvis vi splitter den op i dens dele bliver den meget nemmere at læse og forstå.

1. `^([sS])$`
2. `^([mM])$`
3. `^([lL])$`
4. `^[Xx]+([Ss])$`
5. `^[Xx]+([Ll])$`
6. `^[0-9]{1,2}[xX]([Ss])$`
7. `^[0-9]{1,2}[xX]([Ll])$`

Mellem hver af disse expressions er der en "|" tegn, som nævnt før betyder det "eller". Det er det der gør at man kan skrive mange små sammen til en stor. Som nævnt tidligere er dette ikke helt det samme som en OR operator i normal kodning, da hver regex bliver kørt på samme tid. Det betyder at den prøver mange forskellige igennem på samme tid. Så snart en af dem fejler fortsætter den med resten indtil alle har fejlet, eller en har et fuldt match.

For at forklare de 7 forskellige regex, kan vi først snakke om hvad der er til fælles. Hver regex starter med et "^" symbol, og de slutter alle med en "\$" symbol. Det er for at optimere vores regex så meget som muligt. Det første symbol betyder "i starten af teksten". Og det andet betyder "i slutning af teksten". Hvad det betyder i praksis er at den leder hele teksten igennem, og hvis den ikke finder en meget specifik streng, fejler den med det samme.

Men hvad leder den efter? Tanken er at mange forskellige størrelser kan komme ind forbi programmet. Så der er ingen garanti på om der kommer en størrelse igennem, (som ville være tal. 36-45 oftest) eller andre slags input. Så de her regex leder efter om det er en T-shirt størrelse. I dem er der small medium og large, repræsenteret med S M eller L. Det er hvad de første 3 leder efter. Så hvis vi forklarer en, har vi forklaret alle 3. Så lad os kigge tættere på den første.

Det som skal give et match i de første 3, er enden "S" "M" eller "L" uden nogle mellemrum eller bindestreger og lignende. Den første af de 3 regex er til at håndtere small. Den første i listen har de to symboler vi nævnte før, som betyder at det skal være hele teksten. Så har den et par parenteser "()", og inde i dem firkantede parenteser "[]".

De normale parenteser er en "Capture Group". Det er så at vi ved hvad i figur 2 regex som har fået et match. Regex kører hele teksten igennem, og hvis den første matcher, så har Capture Group 1 et match. Vi kan så igennem kode tilgå denne værdi. Da regex returnere alle Capture Groups i et Array, betyder det at hvis det var et match, er der en værdi. De forskellige Capture Groups stiger i Index, efter deres rækkefølge de bliver introduceret i den individuelle regex (Capture Group 0 er altid den fulde streng der indeholder et match). Så vi har en Capture Group. Men hvad er så de firkantede parenteser inde i denne Capture Group?

De firkantede parenteser betyder "Led efter alle tegn der er her inde". Hvis man skriver "s" inde i denne firkant, betyder det at Regex leder efter et lille s. Hvis man gerne vil have både store og lille s skal man skrive begge. Det er også det der er inde i den første regex.

Så hele den første Regex siger "Alt teksten skal indeholde, er enden et stort eller et lille s. Hvis den har det, så sætter vi Capture Group 1 til at have et match, med værdien af den tekst som blev matchet" Det er det samme i de 2 andre (nummer 2 og 3 i listen). De leder på samme måde efter en tekst der kun indeholder M eller L.

Jeg har splittet dem op i 3, for at undgå at skulle bruge "*" tegnet i Regex. Den siger nemlig "nul eller mange". Hvad den gør, er at den går en ad gangen hen ad hele teksten, og ser om der er noget der passer. Det ville være vanvittigt langsomt hvis en større størrelse kom ind i systemet, så som "EU 39/40 - US 8/9" som er en sammenligning mellem europæisk og amerikanske skostørrelser. Hvis vi kan undgå at gennemgå hver karakter og tilbage igen ville vi spare meget tid. Det er dermed værd at splitte dem op i 3, selvom det måske gør det sværere at læse og forstå. Og vi ville skulle bruge "*" tegnet hvis vi stadig gerne ville vide om vi har s m eller l at gøre med. Den ville se sådan ud "^([sS])*([mM])*([lL])*\$". Jeg har også testet tiderne på denne version, samlet med den version jeg bruger, og i værste tilfælde tager det 12 steps i min version til at finde en L, og kun 6 for S (der er flere da den er senere i listen), Men vigtigst hvis det ikke er nogen af dem tager det kun 9 steps at fejle. Hvor denne version med "*" tegnet, ville kunne tage uendeligt, hvis der var en masse "M" i steg, eller lign. Samt er der også mulighed for at få flere matches, hvis en streng hedder SML vil den matche alle 3 på samme tid.¹

Så det var de første 3 ud af 7. Nu har vi bare 4 tilbage, og af de 4 er der kun 2 forskellige slags, da vi igen splitter L og S fra hinanden. Alt herfra er nemlig med "ekstra" store eller små. Og siden man ikke kan have "ekstra medium" har vi kun 2 forskellige cases for Small og Large.

¹ [1]

Denne regex, "`^[Xx]+([Ss])$`" har nogle af de samme symboler som de første 3. Nemlig `^` og `$` i starten og slutningen. Samt har den en Capture Group der fanger `s` eller `S`. Men hvad der er nyt ved denne, er at vi også skal have `x` eller `X`. Det denne skal matche er ting så som `"XS"` eller `"XXS"` osv. Der er ikke noget limit til hvor mange `x`'er der kan være, men praktisk set ser man aldrig mere end 3. Men da der kan være flere, så har jeg valgt at bruge `"+"` symbolet. Det betyder "en eller mange". Så vi siger at der skal være minimum et `x`, og gerne flere. Efterfulgt af et `s` før dette er et match. Vi har ikke en Capture Group på `X`'erne, da jeg senere i koden har en Regex til at fange dem. Jeg kunne også tage dem her, men i koden på dette tidspunkt er vi ikke interesseret i antal `X`'er, så det ville være forvirrende hvis vi tildelte en masse variable allerede her. Jeg har så valgt at køre Regex en gang til senere for at få den information jeg har brug for, men bare senere så jeg vedligeholder noget læsbarhed i koden. Det fandt jeg også ud af da jeg selv skulle læse koden igen, at det var svært at holde styr på da det blev meget komplekst.

Den sidste del af regex i den samlede er så `^[0-9]{1,2}[xX]([Ss])$`. Her har vi den samme slutning som den før. Vi leder dog kun efter et `X`, og så efter `s`. Før det har vi det nye. Vi har en firkantet parentes som indeholder `"0-9"`. I stedet for at vi leder efter `0`, - og `9`, så leder vi efter alt mellem `0` og `9` i unicode. Så man kan også sige `A-z`, så den tager alt mellem stort `A`, og lille `z`, som er alle store bogstaver, efterfulgt af alle små bogstaver.

Symbolet `{}` betyder "Gentag det tidligere token dette antal gange", hvor "token" er hvad der er i firkantede parenteser. Vi siger her 1 eller 2 gange. Så vi skal have enden 0 til 9 mindst 1 gang, og max 2. Det betyder at vi har en grænse på `"99XL"` og alt over ville ikke matche. Tanken er at intet tøj går så højt op, og ved at sætte en grænse på det kan vi være mere specifik, og dermed gør den hurtigere. Hvis vi i stedet havde `"+"` ville vi kunne have uendelige store extra small, og det ville så tage en del tid at fejle hvis systemet ren faktisk fik et stort tal ind.

Når vi så sætter alle disse 7 forskellige regex sammen ender vi med den regex i Figur 2, hvor man nu kan forstå hvad den leder efter. I mit program har 8 forskellige regex, nogle lidt simplere, men mange i samme kompleksitet som den vi har gået igennem, samt den i Figur 1. Jeg kunne bruge langt tid på at forklare hver af dem i detalje, da jeg er endt med at blive meget glad for Regex. Men vi forstår nu hvad Regex er, og vi har dermed ikke grund til at kigge på hver især.

Valley af Tal / Custom Hasing

I sorteringstrinnet, skal vi på en eller anden sortere strenge med forskellige betydninger, og længder. Den måde jeg valgte at gøre dette på, var at oversætte de strenge, til unikke værdier der stiger i værdi. Til det formål krævede det en custom løsning til hver "protokol", hvor en protokol er hver unik løsning til hver slags tøjstørrelse.

Det ville være umuligt at sortere alle slags tøj på en gang, da man ikke kan oversætte en skostørrelse til en T-shirt størrelse. Så for at fixe det problem sørger vi for at vi kun sorterer den samme slags tøj ad gangen. Så vi kører sorteringen en gang for T-shirts, og en gang for sko osv. Dette gør at vi bare skal mappe en protokol ad gangen, hvor vi bare skal sørge for at det virker internt i hver protokol, og ikke på tværs.

Måden det blev lavet på hver protokol er meget ens, men har nogle forskelle dele for at få dem til at virke. Lad os kigge på T-shirt, og BH størrelser da de virker på forskellige måder, og viser de forskellige metoder jeg har brugt til resten af protokollerne.

```
if (!string.IsNullOrEmpty(amountOfExtra.Groups[1].Value))
{
    //nX
    CM.SortingIndex = IndexValue * 10000 + (double.Parse(amountOfExtra.Groups[1].Value) * 1000 + (-1 * SorL)) * SorL;
    sortList.Add(CM);
}
```

Figur 3 - T-shirt størrelse sort nX linje 101-106 i sortBySize

I figur 3 kan man se det regnestykke vi bruger til at oversætte en streng til et tal der stiger i værdi efter størrelsen. Den hurtige forklaring er at man først sætter en "grund" værdi, der styrer om det er Large Medium eller Small. Vi sætter tidligere IndexValue til at være 3 hvis det er large, 2 hvis det er medium, og 1 hvis det er small. Vi ganger så 10.000 på dette, så vi "siger" til programmet at dette er vigtigst at sortere på. Dernæst tilføjer man ekstra værdier som gør tallet større, som kommer fra hvor meget "Ekstra" der er i størrelsen (3xl vil være 3 ekstra). Vi ganger dette med 1.000, så vi igen "siger" til programmet, at dette er næst vigtigst. Vi gør dette for det sidste trin, som gør at XXL kommer efter 2XL, bare så at der er et system til det, og de ikke skifter frem og tilbage. Måden vi gør dette på, er at vi tilføjer enden +1 eller -1 angående om det er small eller large (Da 2xl er større end xl, men 2xs er mindre end xs, skal vi "flippe" om det er plus eller minus 1 for at vedligeholde rækkefølgen). Til sidst ganger vi -1 på hvis vi det er en small, så vi trækker tal fra, angående efter hvor meget ekstra der er på størrelsen.

```
//
//
CM.SortingIndex = { int.Parse(braRegex.Groups[1].Value) * sortByFirst } + { ((int)braLetter[0]*100 + (braLetter.Length - 1) + HigherSize) * sortBySecond; }
sortList.Add(CM);
```

Figur 4 - BH størrelse sort linje 326-329 i *sortBySize*

I Figur 4 har vi så BH-størrelsen, og man kan måske se noget ens i de to, men der er stadig forskelle. I denne har vi mulighed for at sortere efter første eller anden værdi. Vi oversætter dem stadig begge to til en værdi. Den første del indeholder omkredsen, vi parser den til et tal og ganger de 10.000 eller 1 på.

Dernæst gør vi noget anderledes. Vi siger "(int)braLetter[0]". Det vi gør her er at vi *caster en char til en int*. Hvad det gør, er at den omdanner den givne chars værdi til den Unicode tal. Tallene stiger alfabetisk, så dette ville automatisk gøre B mindre end C osv. Vi ganger dette med 100, for at få en "Valley" at smide andre tal ind i.

Vi ligger så antallet af bogstaver i størrelsen ind i denne Valley. Så hvis det er "BB" så er det større end B, men stadig mindre end C. Selvom de er samme størrelse, gør vi stadig BB mindre så de altid dukker op før C, så de igen ikke er tilfældige hvad der sker først.

"HigherSize" er måske navngivet lidt dårligt, da den altid er 0, med mindre at vi har flere "A" størrelser, da AA er mindre end A. Hvis det er A, så er den – antallet af ekstra bogstaver.

Helt til sidst ganger vi sortBySecond. Den er normalt 1, så den gør ikke noget normalt. Men hvis man har sat den til at sortere efter anden værdi er den 10.000 i stedet for sortByFirst. Det gør så at alt der har noget at gøre med kop-størrelse er vægtet højere end omkredsen.

Procesbeskrivelse

For at arbejde med programmet på en effektiv måde, kræver det at man har en bevist plan om ens proces. Dette kan være at man laver en tidsplan, eller at man kører en specielt projektstyrings værktøj så som Scrum eller RUP. Når man arbejder med flere mennesker kræver det at alle er på samme side over hvordan noget skal gøres. Da folk kan have forskellige metoder at arbejde på, samt forskellige tidspunkter at arbejde på, kan det være svært at være enige om hvad man skal gøre, og hvornår.

Scrum

Man kan også tage de dele som man har behov for fra de forskellige projektstyringsværktøjer. Før i tiden har jeg fået meget værdi ud af Scrums "Daily Standup", som går ud på at man tager et kort møde hvor man står op, (for at holde det træls så det ikke varer for lang tid) hvor man snakker om hvor man er, om der er nogle problemer osv. Scrum er designet til grupper på 3-9 mennesker, men man kan godt adaptere det til en enkeltmands gruppe også. Man har så bare 3 "hatte" på selv. Man er nemlig både "produkt owner" "Scrum master" og udvikler. Så man skal både tænke på "Hvad skal laves", samt "hvordan skal det laves" og så ren faktisk lave det. Det kan være en måde at tvinge sig selv til at fokusere på vigtige features, i stedet for at lave hvad man selv har lyst til på tidspunktet. Det kan dog være lidt svært at holde disse roller separat, da det læner ret meget på sin egen selvkontrol. Tanken kan være god, og når man siger til sig selv "fokuser på vigtige features i stedet de sjove at lave" har man produkt ejer hatten på. Så det er ikke fordi man skal leje forskellige personer, men bare gøre deres "job" for en selv.

Sprints

Man kan også bruge sprints fra Scrum for en en-mans gruppe. Det kan være smart at gøre, da det giver en deadlines tidligere i projektet, som kan være en god motivation. Når man har en lang deadline, og man ikke har det klareste billede om hvor lang tid ting vil tage, kan man nemt få tanken "Det skal nok gå. Jeg har masser af tid". Men dette er en fælde mange falder i, og ender med at man laver det meste af arbejdet lige op til den rigtige deadline. Så hvis man laver sprints til sig selv, giver man deadlines til motivation, samt en plan at følge. Der er andre værktøjer til at have en tidsplan, så som en "Gantt Chart" som man nok tænker oftere på når man tænker på en tidsplan for et projekt. Forskellen på disse metoder, er dog at man ikke splitter sin tidsplan op i alt for små dele. Hvis man i sin Gantt Chart splitter sine opgaver i for små dele kan man komme til at have en tanke om at man er foran, og dermed slapper mere af da man kan bruge det tid man har sparet nu, senere hen. Med sprints er der grænser for løs man kan være. Næste sprint er stadig en til to uger i længde, det er lige meget hvor hurtig man var før. Det er for dette grund at jeg havde tænkt at bruge Sprints selv, i stedet for et Gantt Chart.

Retrospektiv

En vigtig del af Scrum er et retrospektiv efter hvert sprint. Det er et vigtigt værktøj som man kan bruge til at tage vigtige ændringer hvis noget gik galt i ugen før. Det er nok også det bedste værktøj i Scrum til en single udvikler, da man nemt kan blive distraheret uden andre at holde en på spor. Hver uge kan man tage 15 minutter til at spørge sig selv, "Hvorfor arbejde jeg ikke så meget på mit projekt i denne uge?" hvor man kunne finde ud af at man ikke skal arbejde i sin seng, da man bliver træt og mindre effektiv. Det at have tid sat af til at reflektere over sin proces tidligt og ofte, gør at man altid er på sporet, og arbejder effektivt. Jeg ville have disse retrospektives med, men fandt dog ud af at det kan være svært ikke bare at sige "Det gik godt" eller ikke at komme med undskyldninger under denne tid. Så jeg startede med at bruge retrospektives, men endte ikke med at fortsætte med at bruge dem.

Kanban Board

Jeg kunne også have valgt at lave et "Kanban board", hvor man skriver alle de opgaver man har, og opdeler dem efter hvad der skal laves, hvad der er i gang, og hvad der er færdig. Samt kan man sortere dem på farve til at vise de forskellige teknologier der er i dem, eller hvilken rækkefølge man skal gøre dem i. Dette er en meget god løsning, hvis man har mange systemer der skal laves, hvor de ikke er alt for tæt koblet sammen. Et eksempel på dette kunne være hvis man laver et spil, hvor 3D modeller, animationer, og koden kan laves separat. De skal alle gøres, men de har ikke noget med hinanden at gøre, før det sidste trin hvor man sætter dem sammen. Da jeg tænkte over hvilket projekt jeg skulle lave, og kom i tanke om dette, gik det op for mig at det ville være meget lineært. En ting sker, som gør den næste starter osv. Jeg tænkte dermed at et Kanban board nok ikke gav alt for meget mening for mit projekt.

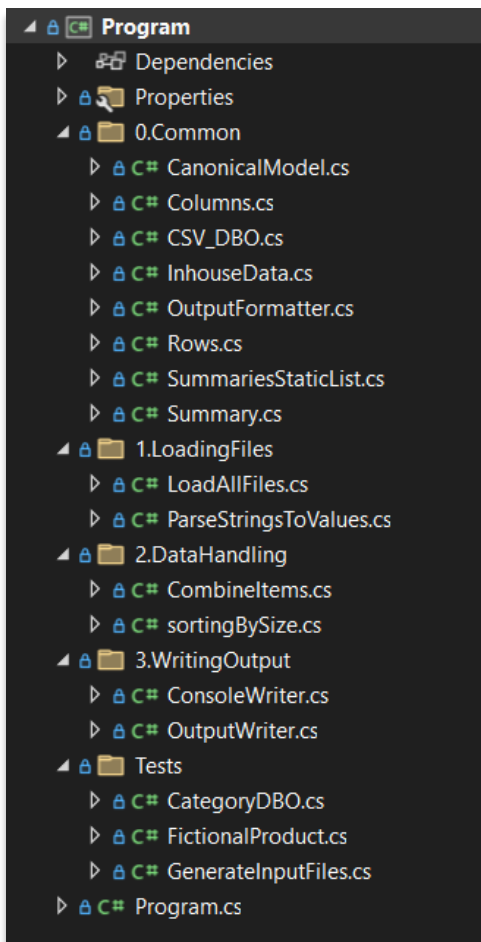
Vandfald

Vandfalds modellen er nok den nemmeste projektstyrings værktøj at bruge. I stedet for at springe frem og tilbage mellem forskellige dele af processen, som man gør i for eksempel RUP, gør man en del færdig, før man begynder på den næste. Jeg tænkte at dette var ret simpelt, og ville passe mit programs struktur meget godt. Så jeg havde dermed valgt at bruge vandfalds modellen til mit program. Jeg tænkte alle de forskellige dele igennem før jeg begyndte at skrive, og da jeg så begyndte, skrev jeg programmet i den rækkefølge som en fil ville komme igennem det. Startede med input, så omskrivning, så data håndtering og til sidst output. Det gjorde det nemt at forstå hvor i processen jeg var, og gjorde at jeg havde en god fornemmelse af hvor langt i projektet jeg var.

Produktbeskrivelse

Selve produktet er et program som er designet at køre en gang, hvor den kan tage så mange filer ad gangen man giver den. Hvordan det fungerer er både simpelt, og komplekst på samme tid. For at komme igennem det kan vi starte med fil strukturen som vi nævnte tidligere. Vi kommer til at kigge ind i de forskellige klasser senere, men vi starter lige med et overblik.

Fil Struktur



Figur 5 - Fil struktur

Når vi kigger på Figur 5, kan vi se alle de forskellige klasser i programmet. Jeg nævnte tidligere opdelingen af disse filer, for at ligne GoFacts' filsystem, men her kan vi se det.

Vi har opdelingen af et nummer, samt et navn der beskriver hvad der sker. Nummeret er for at vi starter med det første der sker i toppen, og så kan vi gå længere ned af listen for at komme længere ind i programmet.

Til at starte med har vi common. Det er alle filer som oftest bliver refereret af alle dele af programmet. Her indeholder ting så som vores "InhouseData" som indeholder alt oversat data fra en CSV-fil, oversat til C# objekter med de rigtige typer for hver felt. Hver klasse i steps 1-3 referere til Common.

Step nr. 1 er så "LoadingFiles". Det er her at vi håndterer at indlæse data fra en CSV-fil, samt oversætte den til de rigtige data typer.

Step nr. 2 er "DataHandling". Det er her vi laver om på dataen. Det er her vi komprimerer / opsummerer kundes data, samt sortere deres data.

Sidste step, nr. 3 er "WritingOutput". Det er her vi har klasser som skriver feedback til konsollen, og udskriver resultatet til både CSV og til JSON.

Jeg har også en mappe til test data, som også burde inkludere unit tests, men det fik jeg ikke lavet i tide. I test mappen har vi i stedet klasser som laver fiktive data hver gang produktet kører. Hvis vi havde det samme data vi brugte hver gang, kunne man have mistænker om at det ikke ren faktisk gjorde hvad vi siger det gør. I vores test mappe har vi en DBO der indeholde de potentielle data et produkt kunne have. Den er her i stedet for common med alle andre DBO'er, da det kun er de klasser i samme mappe som tilgår denne. Resten af programmet ved ikke at den eksisterer. "FictionalProduct" er også en DBO, som indeholder hver instans af et fiktivt produkt, med data som er valgt fra "CategoryDBO".

GenerateInputFiles

Selve "GenerateInputFiles" genererer fiktiv data, og jeg tænker vi lige kan kigge på hvordan den gør det.

```
private void MakeFictionalProducts()
{
    // modify ↓ to change total amount of items
    int differentItems = 250;
    for (int i = 0; i < differentItems; i++)
    {
        FictionalProduct nextProduct = new FictionalProduct();

        int currentCategory = rand.Next(0, categoryOptions.Count - 1);
        int minPrice = categoryOptions[currentCategory].PriceMin;
        int maxPrice = categoryOptions[currentCategory].PriceMax;
        int sizeCount = categoryOptions[currentCategory].Sizes.Count() - 1;
        int EAN = rand.Next(100000000, 999999999);

        nextProduct.EAN = EAN;
        nextProduct.ItemGroupName = categoryOptions[currentCategory].ItemGroupName;
        nextProduct.SoldPrice = rand.Next(minPrice, maxPrice);

        nextProduct.BoughtPrice = Math.Round(nextProduct.SoldPrice * double.Parse("0," + rand.Next(70, 90)), 2);
        nextProduct.Size = categoryOptions[currentCategory].Sizes[rand.Next(0, sizeCount)];

        availableProducts.Add(nextProduct);
    }
}
```

Figur 6 - GenerateInputFiles linje 52-75

Dens funktion er primært splittet mellem 2 funktioner. Vi har den i Figur 6, som laver en masse forskellige produkter ud fra et array som indeholder alle mulige værdier. Vælger en tilfældig kategori at lave et produkt i, og laver en tilfældig EAN-kode. Dernæst laver vi en tilfældig salgspris og laver købt prisen mellem 10-30% mindre. Vi vælger også en tilfældig størrelse. Vi gemmer disse fiktive produkter i en liste, så vi senere kan bruge disse fiktive produkter.

Det array som værdierne tager fra, er længere oppe i koden, og kan ses i Figur 7. Vi laver et array med hver forskellig kategori, samt alle de forskellige størrelser den kunne vælge. Vi sætter også en minimum og max pris på hver kategori.

```
new CategoryDBO("Top", 20, 80, ["3XS", "2XS", "XXS", "X242S", "S", "M", "L", "XL", "2XL", "3XL", "4XL"]),
new CategoryDBO("Pants", 40, 120, ["28W/30L", "30W/32L", "32W/32L", "34W/34L", "36W/34L", "38W/36L"]),
new CategoryDBO("Socks", 5, 15, ["One Size", "25-30", "30-36", "36-45"]),
new CategoryDBO("Bra", 30, 70, ["32A", "32BB", "32B", "32C", "36C", "36DD", "36D", "40D"]),
new CategoryDBO("Jacket", 100, 450, ["S", "M", "L", "XL", "2XL"]),
```

Figur 7 - GenerateInputFiles linje 23-27

```

private void GenerateCSV(string folderPath)
{
    string headers = "EAN;Sales;Size;SoldPrice;BoughtPrice;ItemGroupName\r\n";

    //writingData
    string filePath = folderPath + "/DummyData.csv";
    File.WriteAllText(filePath, headers);
    List<string> lines = new List<string>();
    for (int i = 0; i < finalLineCount; i++)
    {
        int productNumber = rand.Next(0, availableProducts.Count);

        int salesQuantity = rand.Next(0, 10);
        int EAN = availableProducts[productNumber].EAN;
        string Size = availableProducts[productNumber].Size;
        double soldprice = availableProducts[productNumber].SoldPrice;
        double boughtPrice = availableProducts[productNumber].BoughtPrice;
        string itemGroupName = availableProducts[productNumber].ItemGroupName;

        lines.Add($"{EAN};{salesQuantity};{Size};{soldprice};{boughtPrice};{itemGroupName}");
    }
    File.AppendAllLines(filePath, lines);
}

```

Figur 8 - GenerateInputFiles linje 80 -104

På Figur 8 har vi den anden metode som ren faktisk laver CSV filen som programmet bruger som input. Den spændende del er at vi tager et tilfældigt produkt fra den liste vi lavede i Figur 6, og tilføjer alle de værdier til en liste af strenge. Vi skriver så alt dette til en fil når vi er nået til de mængdelinjer man vælger.

Så efter dette trin har vi en CSV-fil med fiktive produkter, som har samme værdier som gør at vi kan tælle dem sammen som et produkt. Efter det er kørt har denne klasse intet mere at gøre, da dens tur til at lege med daten er slut. Vi går så direkte videre til næste step, hvor vi kigger på de spændende dele af indlæsning af filerne.

Step 1. Loading Files

For at læse CSV filen vi får fra vores fiktive data generation, eller fra en kundes egen data, skal vi kunne splitte CSV filen op i individuelle værdier. For at gøre dette skal vi vide hvilken delimiter som CSV filen bruger. For at finde ud af det har jeg skrevet metoden i Figur 9. Den har et array af mulige karakterer som kan være en delimiter. Vi går dernæst igennem den første linje i teksten, og tæller antal af gange det symbol dukker op. Det symbol som der er oftest brugt, bliver sat til at være vores delimiter.

```
private char FindSeperationChar(string RowOfCSV)
{
    char[] possibleSeparators = [
        '\t',
        '\n',
        '|',
        '\\',
        '/',
        '\r',
        ';',
        ' '
    ];

    int highest = 0;
    char separatorCharInTheCSV = '.';
    foreach (char c in possibleSeparators)
    {
        string Pattern = $"\\{c}";
        int count = Regex.Count(RowOfCSV, Pattern);
        if (count > highest)
        {
            highest = count;
            separatorCharInTheCSV = c;
        }
    }

    return separatorCharInTheCSV;
}
```

Figur 9 - LoadAllFiles linje 57-83

Når vi så har vores delimiter, kan vi splitte den første linje for at få vores header værdier. Dernæst læser vi resten af filen, hvor vi tilføjer hver værdi til en liste af strenge, på et DBO der hedder "Rows". Vores slutresultat efter dette, er en liste der indeholder både header værdierne, samt hver eneste værdi i kroppen i sin egen entre i en liste af strenge. Vi gør denne proces for hver fil der er i input mappen, og når vi er færdige, returnerer vi så en liste der indeholder hver fils data "mapped".

```
//Reader from CSV
string line = reader.ReadLine();

char delimiter = FindSeperationChar(line);
foreach (string item in line.Split(delimiter))
{
    CSVData.HeaderValues.Add(item);
}

//data from CSV
while (!reader.EndOfStream)
{
    line = reader.ReadLine();
    Rows newRow = new();
    foreach (string column in line.Split(delimiter))
    {
        newRow.Columns.Add(column);
    }
    CSVData.AllRows.Add(newRow);
}

//from here, the CSVData variable should contain all the data from the CSV
AllDataFilesLoaded.Add(CSVData);
}

return AllDataFilesLoaded;
```

Figur 10 - LoadAllFiles linje 30-54

Når vi så har indlæst hver fil i den forkerte data type, skal vi oversætte dem til de rigtige typer. For at gøre det har vi den metode i Figur 11 der tager hvad vi fik fra Figur 10, og oversætter dem til de rigtige typer. Det er også her vi ser hvorfor man skal oversætte header navnene i input filerne, da vi har de specifikke strenge som vores cases i en switch. Det spændende er at vi

```
public InhouseData ParseValues(CSV_DBO data)
{
    //CanonicalModel is the data from a CSV file in the correct data types
    InhouseData ParsedCSV = new();
    ParsedCSV.FileName = data.FileName;
    for (int i = 0; i < data.AllRows.Count; i++)
    {
        CanonicalModel CM = new();
        for (int j = 0; j < data.HeaderValues.Count; j++)
        {
            if (data.HeaderValues.Count != data.AllRows[i].Columns.Count)
            {
                ConsoleWriter.Write("File ;" + data.FileName + "; has more data rows than header rows.", ConsoleColor.Yellow);
                ConsoleWriter.Write("; Skipping File!", ConsoleColor.Red);
                return null;
            }
            switch (data.HeaderValues[j].ToUpper())
            {
                case "EAN":
                    CM.EAN = int.Parse(data.AllRows[i].Columns[j]);
                    break;
                case "SIZE":
                    CM.Size = data.AllRows[i].Columns[j];
                    break;
                case "SOLDPRICE":
                    CM.SalesPrice = Decimal.Parse(data.AllRows[i].Columns[j]);
                    break;
                case "BOUGHTPRICE":
                    CM.BoughtPrice = Decimal.Parse(data.AllRows[i].Columns[j]);
                    break;
                case "SALES":
                    CM.QuantitySold = int.Parse(data.AllRows[i].Columns[j]);
                    break;
                case "ITEMGROUPNAME":
                    CM.ItemgroupName = data.AllRows[i].Columns[j];
                    break;
                default:
                    Console.WriteLine("Something unknown found in file. Proceeding anyway");
                    CM.Unknown.Add(data.AllRows[i].Columns[j]);
                    break;
            }
        }
        ParsedCSV.Row.Add(CM);
    }
    return ParsedCSV;
}
```

Figur 11 - ParseStringsToValues linje 19-63

har et dobbelt loop, hvor vi kører for hver linje i en CSV, og hver header værdi. Dette betyder at vi kommer forbi hver eneste værdi i en CSV-fil, og dermed kan vi oversætte hver eneste værdi en ad gangen. Der er nogle problemer med tilgangen brugt i Figur 11, som primært er at navne skal passe direkte, ellers bliver værdier ikke oversat. Dette ville være et kæmpe problem i GoFacts' hovesystem, da deres kunders headers kan være lige hvad de havde lyst til, samt være i hvilken rækkefølge de havde lyst til. I stedet for at bruge hardcoded værdier har de et helt andet system som laver en mapping af værdierne til kolonne tal. Det ville ikke virke i vores tilfælde, da hele pointen er at vi er hurtigt ude ved en kunde, og kan køre deres filer igennem med minimale ændringer.

Step 2. Data Handling

Vi har kigget på de vigtige dele til at indlæse data, men det er også bare starten af programmet. Her i step 2 kommer de mere vigtige dele, og de mere komplekse dele. Vi begynder nemt med at se på hvordan vi kombinerer de forskellige produkter til kun en linje, som indeholder alt dens data.

```
public InhouseData Combine(InhouseData FullParsedCSV)
{
    //each row of the CSV
    foreach (CanonicalModel CM in FullParsedCSV.Row)
    {
        if (UniqueProducts.ContainsKey(CM.EAN))
        {
            UniqueProducts[CM.EAN].QuantitySold += CM.QuantitySold;
        }
        else
        {
            UniqueProducts.Add(CM.EAN, CM);
        }
    }
}
```

Figur 12 - CombineItems linje 18-31

På Figur 12 kan man se min logik for at sørge for at vi finder hvert unikt produkt, samt tæller hvor mange af dem vi har set. Vi kører hver eneste linje igennem af de originale filer, og ser om vi allerede har set dem. Jeg har valgt at bruge et Dictionary til dette formål, da man hurtigt kan se om ens Dictionary indeholder en nøgle. Hvis vores Dictionary allerede har den EAN kode vi er kommet til, tæller vi antallet af salg på produktet op med hvad der var på denne linje. Hvis det er et nyt produkt, tilføjer vi både koden og produktet til vores Dictionary. Når denne ForEach loop er kørt igennem har vi et Dictionary der indeholder alle produkterne, samt hvor mange der blev solgt i en fil.

```
foreach (var item in UniqueProducts)
{
    CanonicalModel combinedCM = item.Value;
    combinedCM.GrossProfit = (item.Value.SalesPrice - item.Value.BoughtPrice) * item.Value.QuantitySold;
    combinedData.Row.Add(combinedCM);
}
```

Figur 13 - CombineItems linje 35-40

I Figur 13 kan vi så se næste step, hvor vi har en ny liste som indeholder de kombinerede data. Vi tæller også hvor mange penge man tjente på hvert objekt sammen her. Måden vi gør det på er at vi finder profit for et produkt, og ganger med antallet af salg på det produkt. Resultatet er så et komprimeret dataset der indeholder samme data som input filerne, men bare forkortet så vores sorterings algoritme kører mere optimal. Samt viser resultatet af kundens måned i salg.

Når har vi så det data som kunden gerne skulle se, men det er ikke sorteret. Så det data kunden ser er allerede skrevet, men vi skal nu bytte rundt i rækkefølgen de bliver vist, så det giver mening for kunden. I kravene stod der at de både skulle være sorteret efter størrelse, men også salg. For at sortering i størrelse giver mening, sorterer vi også først baseret på kategorien. Så alle T-shirts er sammen, og alle sko er sammen. Grunden til dette er at nogle kategorier deler størrelser. Så hvis vi bare sorterede på størrelser ville vi få et resultat der viser t-shirts, bluser, jakker og andre tops blandet sammen. Det giver ikke det overblik vi er ude efter, så sorteringens prioritet lyder: Kategori, Størrelse, salg.

```
Dictionary<string, List<CanonicalModel>> categories = new();
foreach (CanonicalModel row in compressedFile.Row)
{
    if (categories.ContainsKey(row.ItemgroupName))
        categories[row.ItemgroupName].Add(row);
    else
        categories[row.ItemgroupName] = new() { row };
}
```

Figur 14 - Program linje 73-82

Den første sortering er ret nem. Vi laver en ny Dictionary hvis nøgle er kategorien. Når vi ser den samme kategori, tilføjer vi den række til en liste, som er værdien i vores Dictionary. Hvis det er en anden kategori end vi har set før, tilføjer vi den til vores Dictionary, samt en ny liste der har den row til at starte med. Vi kunne også tilføje den row bagefter, men på denne måde sparer vi en linje.

```
foreach (string category in categories.Keys)
{
    List<CanonicalModel> sortedByCategory = sortBySize.SmartSorter(categories[category]);
    sortedValues[category] = sortedByCategory;
}
```

Figur 15 - Program linje 83-87

Nå vi så har hver kategori adskilt, kan vi se i Figur 15 at vi kalder vores sorterings metode på hver kategori. Måden vi sortere på kategori er dermed at ikke blande dem sammen til at starte med. Når vi har sorteret en kategori, sætter vi en Dictionary med nøglen af kategorien, til at være den sorterede liste. Tanken er at efter alt er sorteret, kan vi samle denne Dictionary til en liste der er sorteret, og skrive den ud til en fil. Men efter dette trin, har vi kun sorteret efter kategori, og størrelse. For også at få sorteret på antal salg inde for hver størrelse, skal vi igennem nogle flere trin.

For at vi for sorteret på salg inden for hver størrelse, gør vi noget lignende hvad vi gjorde for at sortere på hver kategori. I Figur 16 kan vi se en ens struktur som på Figur 14. En If statement der kigger på om vi har set denne størrelse før. Hvis vi ikke har, så tilføjer vi en ny nøgle i dette Dictionary som er størrelsen, med værdien af en ny liste der indeholder den komprimerede information. Vi sætter sorterings indeks på hver genstand til at være hvor mange salg der har været, men som et negativt tal. Dette er for at undgå at skulle skrive en mere metode, som sortere for høj til lav, i stedet for lav til høj. Det vi så gør, er at vi sætter de "høje" værdier til at være "lave" bag scene, så vores sorterings algoritme sortere dem som den plejer, men vi får bare resultatet på en måde der giver mere mening.

Den Foreach statement i Figur 16 er inde i en anden foreach, der gennemgår hver kategori som allerede er sorterede efter størrelse.

```
foreach (CanonicalModel listItem in sortedValues[category])
{
    //if we haven't seen the size before, we make a new entry in the size dictionary
    if (!SalesSortedList.ContainsKey(listItem.Size))
    {
        //we set the sortingIndex to be "-listItem.QuantitySold", as the sorting goes low to high.
        //Where we want this output to be high to low. So we "reverse" the ordering behind the scenes
        listItem.SortingIndex = -listItem.QuantitySold;
        SalesSortedList.Add(listItem.Size, new List<CanonicalModel>() { listItem });
    }
    else
    {
        //same as above, but we don't make a new entry, we just add to the existing size list.
        listItem.SortingIndex = -listItem.QuantitySold;
        SalesSortedList[listItem.Size].Add(listItem);
    }
}
```

Figur 16 Program linje 105-121

```
foreach (string size in SalesSortedList.Keys)
{
    List<CanonicalModel> TempList = SalesSortedList[size];
    sortingBySize.InsertionSortByIndex(ref TempList);

    foreach (CanonicalModel sortedListItem in TempList)
        FinalSortedListISwear.Add(sortedListItem);
}
sortedValues[category] = FinalSortedListISwear;
```

Figur 17 - Program linje 128-143

Når vi så har en adskilt liste af hver størrelse i Figur 16, går vi i Figur 17 igennem hver af dem, og sortere en gang til. Forskellen på denne gang, og den første sortering, er at vi genbruger den metode der ren faktisk gør sorteringen, uden af kalde den del er oversætter værdier til et

sorteringsnummer. Vi kan gøre dette da vi i Figur 16 manuelt satte sorteringsnummeret til at være negativ salg mængde. Når vi har sorteret en størrelse, tilføjer vi resultatet til en liste der kommer til at indeholde alle entries i kategorien. Når alle størrelser er kørt igennem erstatter vi den kategori der var der før, med den nye som er sorteret per størrelse, per salg. Vi kigger på selve algoritmen senere, men først tager vi lige step 3 i kæden.

Step 3. Skrive output

Når vi har sorteret dataen, er vi færdige. Vi skal dermed bare præsentere data til kunden på den måde de kan forstå det. Måden vi gør det på er at udskrive til både CSV og JSON. CSV kender de fleste mennesker, da man kan læse den i Excel. Det er salgs-mennesker der kender dette bedst. Men vi skriver også til JSON. Grunden til dette er at GoFact er et IT-system, og der er en chance for at man skal snakke med kundens IT-afdeling, og ikke med deres salg afdeling eller chef. Vi giver dermed mulighed for forskellige løsninger angående hvem vi snakker med.

```
File.WriteAllText(docPath, headersForFullData);

foreach (var category in outputFormatter.FullDetails.Keys)
{
    List<string> fullData = new();
    foreach (var item in outputFormatter.FullDetails[category])
    {
        fullData.Add(category + delimiter +
            item.EAN + delimiter +
            item.Size + delimiter +
            item.GrossProfit + delimiter +
            item.SalesPrice + delimiter +
            item.BoughtPrice + delimiter +
            item.QuantitySold);
    }
    File.AppendAllLines(docPath, fullData);
}
```

Figur 18 -OutputWriter linje 47-64

På Figur 18 kan vi se hvordan vi skriver til en CSV. Det er en meget nem og manuel løsning, men gør at vi har mere kontrol over hvordan det ser ud. Vi kører en foreach på hver kategori, og skriver resultatet af dette til filen. Vi gør det på denne måde for ikke at have muligheden for at streng listen bliver for lang. Jeg er ikke sikker på om det kunne blive et problem, men tænkte det kunne være smart at undgå det problem hvis det nu kunne komme. Det eneste det koster er lidt performance, da vi skriver flere gange og ikke bare en.

Vi behøver ikke at kigge på JSON output, da vi bare kalder et library til at oversætte et objekt til JSON og skriver det ud. Vi skriver dog begge ud hele tiden, for ikke at skulle have flere sættings til at starte med, det eventuelt kunne skræmme / forsinke brugen af programmet.

Sortering efter størrelse

Vi har nu kigget på hvordan programmet bruger sorteringen, men hvordan virker sorteringsalgoritmen, og hvordan sortere vi tøjstørrelser? Vi har i Teori afsnittet kigget på matematikken på nogle specifikke eksempler, men lad os kigge på hvad der sker hvis vi ikke er sikker på hvad det er.

```
else //any leftover will get sorted with the universal regex. it pretty much only cares about the numbers.
{
    if (!string.IsNullOrEmpty(regMatch.Groups[1].Value) && string.IsNullOrEmpty(regMatch.Groups[3].Value))
    {
        CM.SortingIndex = 10000000;
        sortList.Add(CM);
    }
    //-----
```

Figur 19 - sortBySize linje 344-351

På Figur 19 kan vi se hvor vores "Universal Regex" (UniReg) begynder. Man kan se den i fuld i Figur 1. På dette sted i programmet har den prøvet at sortere efter forskellige slags T-shirts og andre specifikke cases hvor der er bogstaver eller lignende der er vigtige. Så alt efter den "else" i toppen handler om vores UniReg. Måden den fungerer på, er at den har forskellige Capture Groups, og angående efter hvilke er sat, kan vi se om der er 1, 2, 3 eller 4 separerede tal. Oftest ignorerer vi bogstaver og tal, og bruger bare tal værdierne. Vi kommer til at kigge på hver case længere. Men til at starte med tjekker den om der er nogle tal i den, som bliver gjort i Figur 19. Hvis der ikke er nogle tal, er det højst sandsynligt "Onesize" eller lignende. For at vi sortere dem har jeg valgt at gøre dem "størst". Så vi sætter dens sorterings indeks til at være et højere tal end muligt at få normalt, og dermed sikre vi at alle Onesize altid er i slutningen.

```
//-----
// 1 number
if (!string.IsNullOrEmpty(regMatch.Groups[3].Value) //first number
    && string.IsNullOrEmpty(regMatch.Groups[7].Value)) //if 7 is unset, then there's only 1 number in the searchstring
{
    CM.SortingIndex = double.Parse(regMatch.Groups[3].Value) * sortByFirst;
    sortList.Add(CM); //we still add the 1k so it'll play well if there's mixed values
}
//-----
```

Figur 20 - sortBySize linje 352-360

På Figur 20 har vi casen med kun et tal. Vi kan se at vi har en If statement der tjekker hvilke Capture Groups som er sat, og baseret på det ved den hvor mange tal der er. Siden vi ikke kan være 100% sikker på at der ikke er fejl i kundens data, og dermed kommer til at have skiftende protokoller inden for en kategori, tilføjer vi stadig SortByFirst (SBF), som er 10.000 oftest. Efter vi har sat sorterings indeks på hvert objekt, tilføjer vi det til en liste der bliver sorteret senere i programmet. Vi gør det i alle UniReg varianter, så jeg nævner det kun denne gang.

```

//-----
// 2 numbers
else if (!string.IsNullOrEmpty(regMatch.Groups[3].Value) //first number
    && !string.IsNullOrEmpty(regMatch.Groups[10].Value) // after decimal point
    && string.IsNullOrEmpty(regMatch.Groups[8].Value)) // if this is set, then there's 3 or more numbers
{
    double firstNumber = double.Parse(regMatch.Groups[3].Value);
    double secondNumber = double.Parse(regMatch.Groups[10].Value);
    if (Regex.IsMatch(regMatch.Groups[9].Value, @"\.|\,") //if it's a decimal
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value + "." + regMatch.Groups[10].Value) * sortByFirst;
        sortList.Add(CM);
    }
    else if (Regex.IsMatch(regMatch.Groups[9].Value, @"\\|\\/|\\-|\\s*?")) //two ints. example: 27 / 12 or 13 - 21
    {
        CM.SortingIndex = firstNumber * sortByFirst + secondNumber * sortBySecond;
        sortList.Add(CM);
    }
    else
        throw new Exception("Error with 2 numbers: couldn't find endpoint");
    //dont know if these exceptions are needed. they were usefull for debugging
}
//-----

```

Figur 21 - sortBySize linje 360-382

I Figur 21 kan vi se UniReg løsning for 2 tal. Vi sætter først begge værdier til lokale variabler for at gøre det nemmere at læse. Vi tjekker dernæst hvorfor der er 2 tal. Det kan tælle som 2 tal, hvis det er et komma tal, eller hvis det er en rækkevidde. Det gør vi ved at tjekke om det har symbolet ”.” eller ”,” mellem de to tal. Hvis det er en af dem er det et komma tal, og vi oversætter det dermed til 1 decimal tal, og sætter dens sorterings indeks til at være den ene værdi.

Hvis det er en rækkevidde derimod, ganger vi det første tal med vores SBF, og tilføjer den anden værdi ganget med vores SortBySecond (SBS). Resultatet er dermed at vi kan sortere efter det første tal eller andet tal.

Hvis vi ikke kan se om det er et komma tal, eller en rækkevidde, ved vi ikke hvad vi skal gøre med den, så vi laver en Exception.

```

//-----
// 3 numbers
else if (!string.IsNullOrEmpty(regMatch.Groups[3].Value) //first number
    && !string.IsNullOrEmpty(regMatch.Groups[8].Value) //second number
    && !string.IsNullOrEmpty(regMatch.Groups[10].Value) //third number
    && string.IsNullOrEmpty(regMatch.Groups[5].Value)) //checks if 4th number is set
{
    if (Regex.IsMatch(regMatch.Groups[6].Value, @"\.|\,|\/") //if the 1st special character is a decimal, then combine the 1st and 2nd number as the 1st value
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value + "." + regMatch.Groups[8].Value) * sortByFirst + double.Parse(regMatch.Groups[10].Value) * sortBySecond;
        sortList.Add(CM);
    }
    else if (Regex.IsMatch(regMatch.Groups[9].Value, @"\.|\,|\/") //if the 1st special char isn't a decimal point, then if 2nd one is
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value) * sortByFirst + double.Parse(regMatch.Groups[8].Value + "." + regMatch.Groups[10].Value) * sortBySecond;
        sortList.Add(CM);
    }
    else if (Regex.IsMatch(regMatch.Groups[6].Value, @"\\|\/") //if 1st special char is separate value
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value) * sortByFirst + double.Parse(regMatch.Groups[8].Value) * sortBySecond;
        sortList.Add(CM);
    }
    else if (Regex.IsMatch(regMatch.Groups[6].Value, @"\"))
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value) * sortByFirst + double.Parse(regMatch.Groups[10].Value) * sortBySecond;
        sortList.Add(CM);
    }
    else
        throw new Exception("Error with 3 numbers: couldn't find endpoint");
}
//-----

```

Figur 22 - sortingBySize linje 382-415

På Figur 22 ser vi hvad der sker hvis der er 3 tal. Den består af 4 If statements, hvor de første 2 tjekker om det er en decimal værdi mellem det første, eller andet tal. Hvis der er en rækkevidde, tager vi bare det første tal i rækkevidden og "glemmer" top værdien i rækkevidden. Dette kan vi gøre da vi sorterer fra lav til høj. Hvis man sorterede modsat, ville man tage top værdien og glemme bundværdien.

```

//-----
// 4 numbers
else if (!string.IsNullOrEmpty(regMatch.Groups[5].Value)) //if group 5 is set, then there's 4 numbers
{
    //If it's 2 decimals, then we parse them as such
    if (Regex.IsMatch(regMatch.Groups[4].Value, @"\.|\,|\.") && Regex.IsMatch(regMatch.Groups[9].Value, @"\.|\,|\."))
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value + "." + regMatch.Groups[5].Value)
            * sortByFirst + double.Parse(regMatch.Groups[8].Value + "." + regMatch.Groups[10].Value) * sortBySecond;
        sortList.Add(CM);
    }
    //if it's 2 ranges, we only look at the first number in the ranges to sort.
    else if (Regex.IsMatch(regMatch.Groups[4].Value, @"\\|\/|\"))
    {
        CM.SortingIndex = double.Parse(regMatch.Groups[3].Value) * sortByFirst + double.Parse(regMatch.Groups[8].Value) * sortBySecond;
        sortList.Add(CM);
    }
    else
        throw new Exception("Error with 4 numbers: couldn't find endpoint");
}
//-----

```

Figur 23 sortingBySize linje 415-434

Hvis det i stedet har 4 værdier, som er hvor vi kigger på i Figur 23, bliver det meget nemmere for os. Vi kan kigge på om det er komma tal, eller rækkevidder. Der er kun de 2 cases der kan ske, så vi behøver ikke skrive andre muligheder ind i programmet. Hvis det er decimaler laver 2 forskellige decimaler, og ganger SBF og SBS på de forskellige tal. Hvis det derimod er en rækkevidde, ignorerer vi igen top værdien, og bare kigger på de første tal, og igen ganger SBF og SBS på.

```

public static void InsertionSortByIndex(ref List<CanonicalModel> sortList)
{
    bool sorted = false;
    while (!sorted)
    {
        sorted = true;
        for (int i = 1; i < sortList.Count; i++)
        {
            if (sortList[i].SortingIndex < sortList[i - 1].SortingIndex)
            {
                var temp = sortList[i - 1];
                sortList[i - 1] = sortList[i];
                sortList[i] = temp;

                sorted = false;
                //if we swap once, we restart the loop
                break;
            }
        }
    }
}

```

Helt til sidst i klassen har vi hvad der ren faktisk sorterer tallene. Vi kan se indholdet af denne metode i Figur 24. Det er en simpel "insertion sort". Det er en af de simpleste sorteringsalgoritmer at implementere, men den er også hurtig i relative lave tal.

Figur 24 - sortingBySize linje 451-471

Da jeg lavede programmet, tænkte jeg hvor mange elementer vi ville sortere på en gang, og kom frem til at "insertionsort" burde at være mere end fint til dette use-case. Hvis man kigger på programmet på helhed, har vi en blanding på forskellige sorterings algoritmer, da vi splitter dataen op i flere mindre dele fra starten af. Det er hvad andre sorteringsalgoritmer, så som quicksort, gør. Så vi ender med en blanding af forskellige sorteringsalgoritmer ved kun at bruge insertionsort.

Selve sorteringsalgoritmen endte med at være den mest komplekse del af hele programmet, og kræver en del forståelse for at kunne læse og ændre effektivt. Jeg har undervejs tilføjet mange kommentarer, som jeg har prøvet at fjerne fra figurerne, da jeg forklarer hvad koden gør her også, kunne man føle en gentagelse hvis jeg havde beholdt dem i figurerne. Et eksempel på nogle af de kommentarer jeg har efterladt i denne klasse kunne være den på Figur 25, som er en udvidet kommentar til hvad der er på Figur 26.

```

/* <1>
 * This segment is meant to prioritize numbers before raw X's. so 3X is shown before xxx regardless of it's Large or Small.
 * So if it's Large, we "value + (-1 * 1)" = -1 to the value, so it's seen as "smaller" than XXX
 * And if it's small, it adds 1 to the value, so it becomes one larger, so when we flip the value with <2>, then it's also 1 "smaller"
 * <2>
 * We "flip" the value if it's an Small, so that the "biggest" number, like 99XS becomes "-99"xs, and is seen as "smaller" in the
 * sorting algorithm, and will place it before a smaller value
 * and if it's a Large, we dont flip it, and it will just sort smaller first.
 * So <2> is to flip the priority of the sorting algorithm,
 * from "largest" first in the case of "S"
 * to "largest" last, in the case of "L"
 * <3>
 * Parse the value to a double, as it's 5 in "5xl"
 * <4>
 * add 1.000 to the value, so the +-1 difference in 3xl and xxxl made in <1>
 * doesn't change it's position relative to the other values. So 3xl is still considered larger than xxl, even though we -1 from it's value
 * (xxl = 32.000) (3xl = 32.999) (xxxl = 33.000)
 */

```

Figur 25 - sortBySize linje 119-136

```

//nX
CM.SortingIndex = IndexValue * 10000 + (double.Parse(amountOfExtra.Groups[1].Value) * 1000 + (-1 * SorL)) * SorL;
sortList.Add(CM);

```

Figur 26 sortBySize linje 103-105

Man kan se i Figur 26, at der er "tags" over de forskellige dele af matematikken. Det er for at forklarer de komplekse dele i den linje, da det er meget abstract tænkning der skal til for at forstå hvad der sker. Jeg gjorde kommentarerne på denne måde her, og nogle få andre steder i denne klasse, da det ville være svært at give den store forklaring inde i selve koden. Så den kom bare lidt senere nede. Disse kommentarer har været en kæmpe hjælp til mig selv, og for håbeligt også andre, til at huske hvad de forskellige dele gør og er til for.

Resten af programmet har også masser af kommentarer til at hjælpe mig selv og andre der måske skal kigge programmet igennem senere. Der er nok en større end normal mængde af dem, og de kan ødelægge "flowet" en smule hvis man er god nok til at programmere og dermed ikke har brug for dem nær så meget. Men det er bedre at de er der hvis folk, inklusivt mig selv, har brug for dem.

Perspektivering

Under forløbet havde jeg prøvet at gøre ting ordentligt, med tidsplaner, planlagte sprints og faste mål, men det endte med at blive en meget mere kaotisk forløb end jeg havde forventet, da under projektet blev min far ramt af kraft. Det gjorde det meget svært at fokusere på hvordan jeg "burde" arbejde, og gjorde at jeg skulle kæmpe ret hårdt på at bare få arbejdet lidt. Hvis jeg kunne få arbejdet en smule på mit projekt, var det godt nok. Jeg prøvede stadig at holde mig til en nogenlunde tidsplan, men det gjorde at den tidsplan jeg havde lavet til at starte med ikke holdte. Jeg skulle dermed tilbage og genoverveje hvad jeg skulle lave og hvornår. Det endte med den største del af mit arbejde skete hen imod slutningen af forløbet, som jeg havde prøvet at undgå. Grunden til dette var at jo længere tid der gik, jo nemmere blev det at fokusere igen.

Jeg er dog tilfreds med hvor programmet er endt, og jeg føler jeg har kommet i mål med mine krav til programmet. Jeg manglede dog at implementere unit tests, da jeg ikke kunne finde tid til at ren faktisk skrive dem. Der var gange, specielt under udviklingen af min sorterings algoritme, hvor unit tests kunne have været dejligt. Men jeg havde for meget koffein i mig til at have lyst til at gøre ting på den smarte måde.

Hvis jeg havde lavet programmet igen, ville jeg nok have ændret den del. Jeg ville nok også have startet med den mest komplekse del først, så jeg ikke havde så meget stress når jeg kom til den del så sent i forløbet.

Konklusion

Programmet jeg er kommet frem til, indeholder GoFacts' hovedfunktion. Det at optælle, opsummere data og præsentere det på en måde der giver mening. Vi tager enten kundens data, eller fiktive data hvis de ikke gider at bruge deres egen, og kører vores algoritme til at forkorte og sortere dataen. Da man ikke skal bruge mange timer på at manuelt opsætte denne version af deres system, betyder det at vi har en effektiv demo man kan vise til kunder for at sælge dem på ideen. Ved at få flere kunder til at se om GoFact er noget for dem, åbnes døre til mindre kunder at blive kunder også, uden at det bliver et for stor risiko for GoFact.

Jeg har gjort det simpelt at køre, da man ikke skal indtaste nogle specielle indstillinger der ville øge kompleksitet, samt gjort at man bare skal trykke på en knap og bruge computersystemer alle kunder kender til. Det resulterer i en nem oplevelse for kunder, og en hurtig salgs pitch. Dette gør at vi har et program der kan hjælpe kunder med at se GoFacts' værdi, samt gør det at GoFact kan gå efter de mindre kunder, hvor de ikke skal have et stort risiko for at de ender med at tabe penge, da de kunder der ville gå fra ikke går til GoFact efter de har set demoen.

Bibliografi

[1] »regex101,« [Online]. Available: <https://regex101.com/>. [Senest hentet eller vist den 27 12 2025].

Bilag

MoSCoW – Simpel

Must Have

1. Loade en kundes fil nemt
2. Lave udregninger på data
3. Sortere efter industri standart og højeste salg
4. Vise resultatet nemt

Should Have

1. Syntetisk data
2. Support til flere filer på en gang
3. Fejl håndtering
4. Unit tests

Could Have

1. Flere farver I konsollen til at øge læsbarhed
2. En hurtigere måde at ændre filen på en manuel arbejde

Wont Have

1. GUI
2. Advanced GoFact Features